

Formal black box testing

Prof.univ.dr. Tudor Bălănescu

Lect.univ.dr. Cristina Tudose

Comments on the title

- black:
 - poor information about the structure of the implementation under test
- formal:
 - there is a formal specification of the design
 - ⇓
 - test generation is automated
 - successful test implies the implementation is correct*

* As opposite to the famous quote Dijkstra-Burton-Randell (see next slide)

The famous quote Dijkstra-Burton-Randell

- Program testing can be used to show the presence of bugs, but never to show their absence!
 - Source: E. W. Dijkstra: Notes On Structured Programming, T.H. –Report 70-WSK-03, 1970, at the end of section 3, On The Reliability of Mechanisms. [EWD](#)
- Testing shows the presence, not the absence of bugs
 - Source: J.N. Buxton and B. Randell, eds, Software Engineering Techniques, April 1970, p. 16. Report on a conference sponsored by the NATO Science Committee, Rome, Italy, 27–31 October 1969. [BR](#)
(Possibly the earliest documented use of the famous quote)

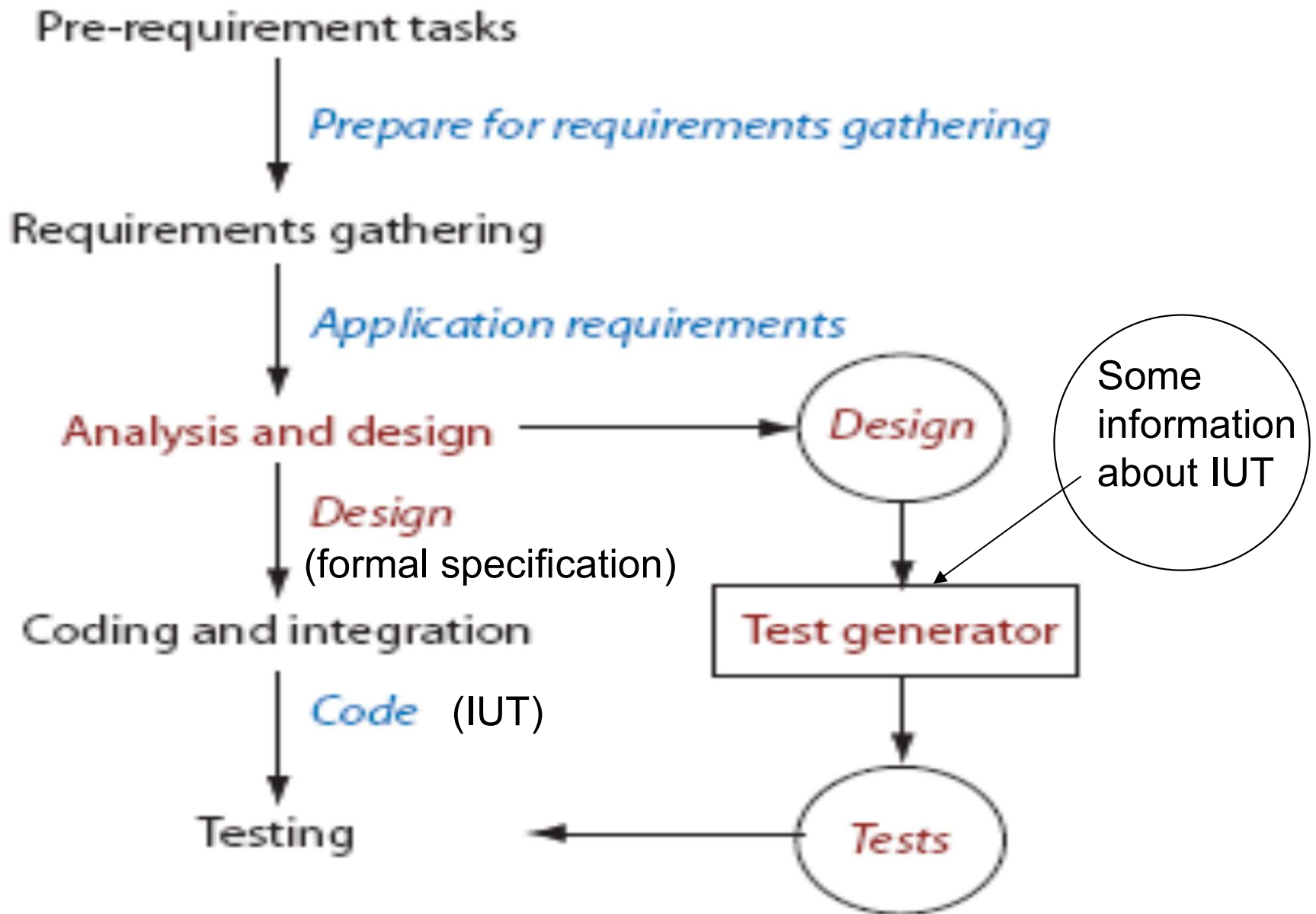
The testing problem

- the design of a system: Finite state machines(FSM), state charts, Petri Nets etc
- IUT: an Implementation(of the design)Under Test
 - a program
 - another design etc.

The testing problem:

to determine if the IUT is *equivalent* to a formal representation of a design

Design and *automated* test generation in a software development process



Test generation procedures to derive tests from FSMs (Finite State Machines)

- algorithms that take a FSM and some attributes of IUT as inputs to generate tests.
- test generation methods can be automated (though only some have been integrated into commercial test tools)

In that presentation:

- the W-method
- the transition tour (TT) method
- the distinguishing sequence (DS) method,
- the unique input/output (UIO) method
- the partial-W (Wp) method.

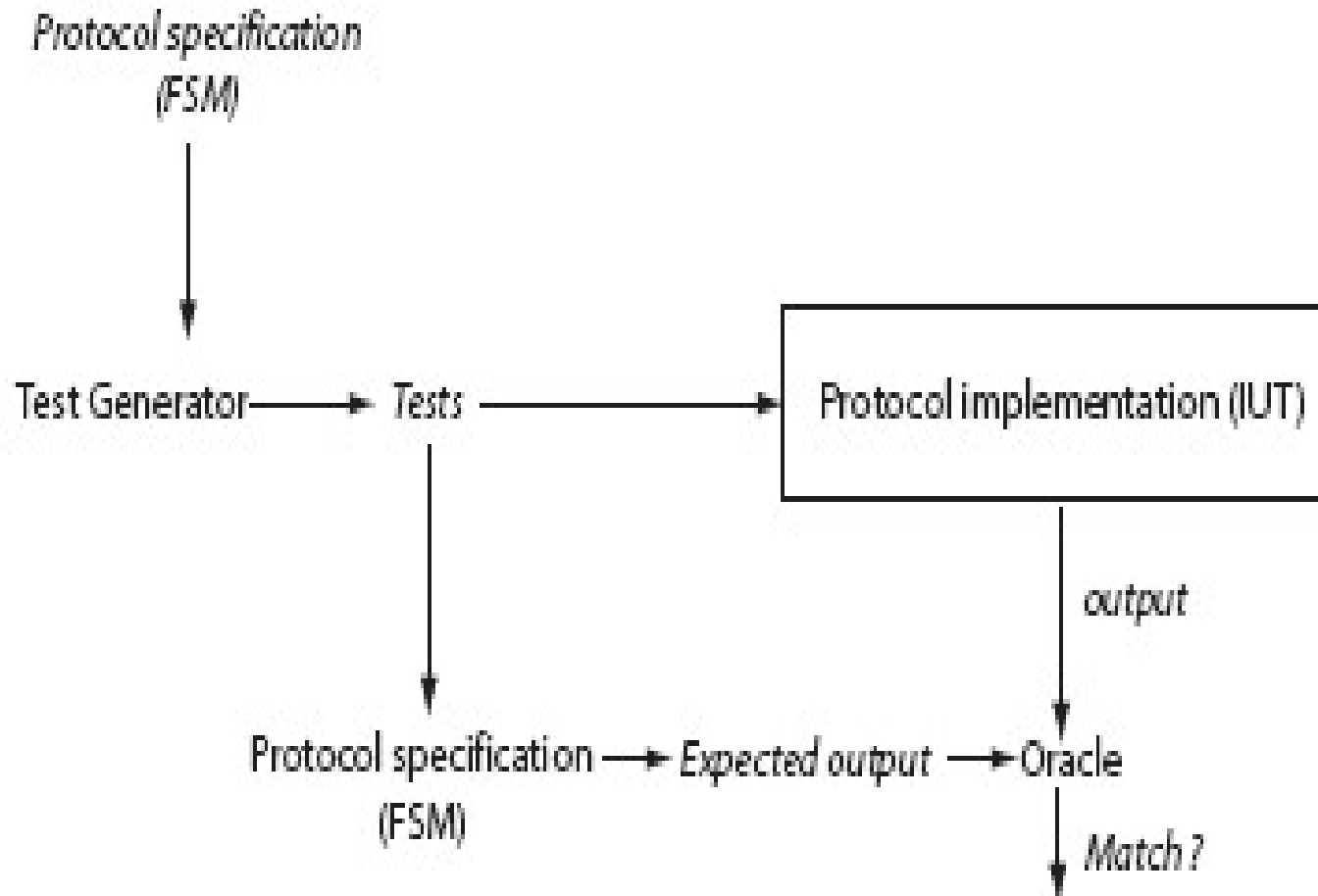
Bibliographic Notes

- the W-method
[Chow78] T.S. Chow, "Testing Design Modelled by Finite-State Machines", IEEE Trans. S.E. 4, 3, 1978.
- the partial-W (Wp) method
S. Fujiwara, G. Bochman, F. Khendek, M. Amalou, and A. Ghedasmi. Test selection based on finite state models. *IEEE Transactions on Software Engineering*, 17(6):591{603, June 1991.
- the unique input/output (UIO) method
K.K. Sabnani and A.T. Dahbura, "A protocol Testing Procedure", Computer Networks and ISDN Systems, Vol. 15, No. 4, pp. 285-297, 1988.
- the transition tour (TT) method
S. Naito and M. Tsunoyama, "Fault Detection for Sequential Machines by Transition Tours", Proc. of FTCS (Fault Tolerant Computing Systems), pp.238-243, 1981.
- the distinguishing sequence (DS) method
G. Gonenc, "A method for the design of fault detection experiments", IEEE Trans. Computer, Vol. C-19, pp. 551-558, June 1970.

Presentation mainly based on:

**Aditya P. Mathur, *Foundations of Software Testing*,
[Pearson Education](#). 2008, 689 pages**

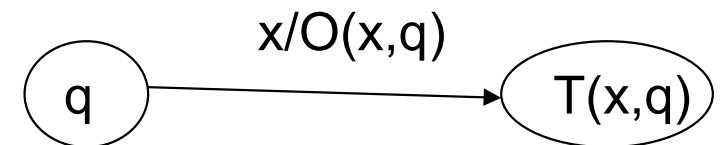
Testing a protocol implementation against an FSM model.



Deterministic FSM (DFSM)

Deterministic FSM:

- $M = (X, Y, Q, q_0, T, O)$
 - X : (finite)Set of inputs
 - Y : (finite)Set of outputs
 - Q : (finite)Set of states
 - q_0 : the initial state,
 - T : Transition function, $X \times Q \rightarrow Q$,
 - O : Output function, $X \times Q \rightarrow Y$.
- T and O are extended, in a canonical way, to domain $X^* \times Q$
 - $T(ax, q) = T(x, T(a, q))$, $T(\epsilon, q) = q$,
 - $O(ax, q) = O(a, q)O(x, T(a, q))$, $O(\epsilon, q) = \epsilon$

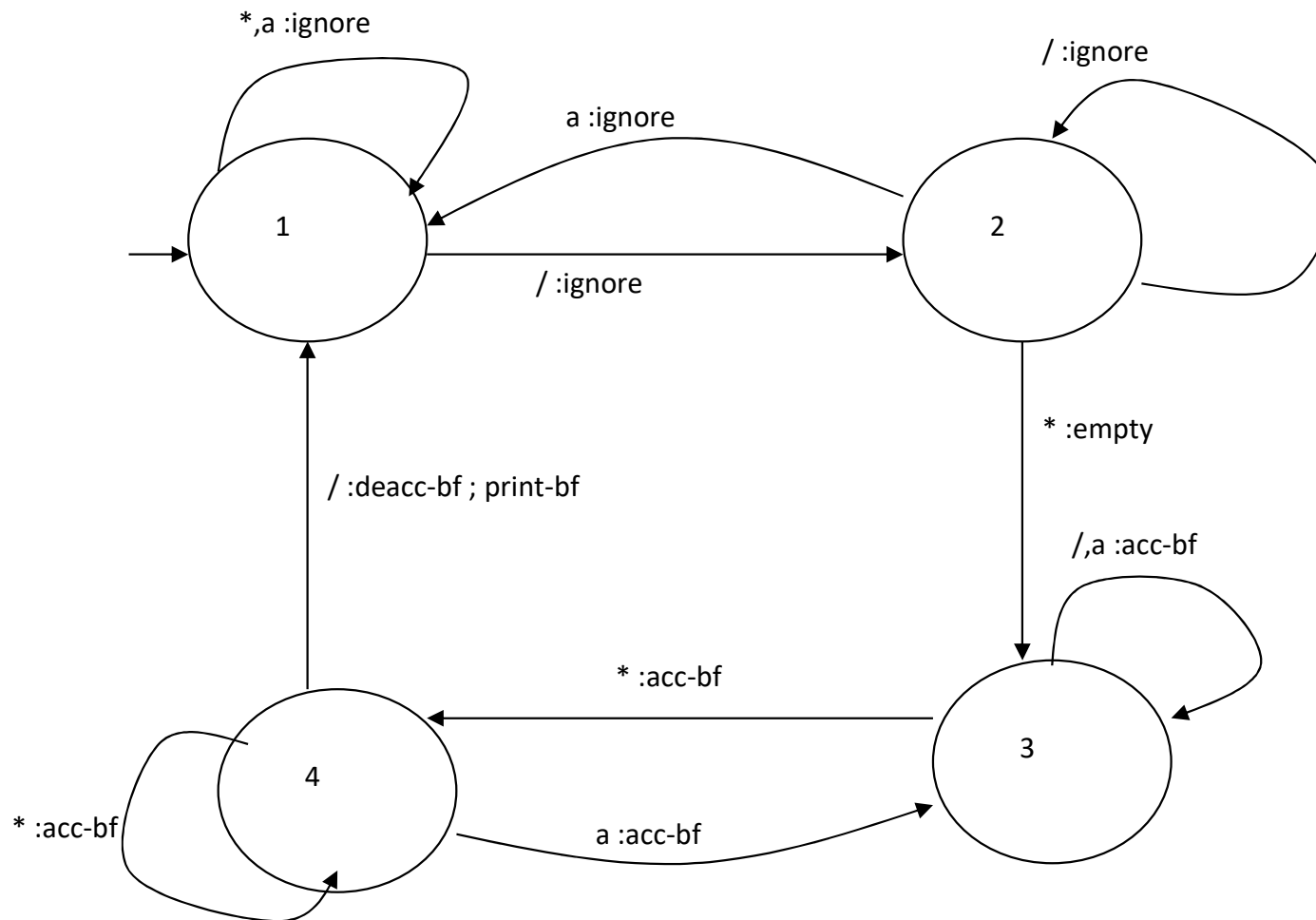


DFSM specification for a “C comments” printing system

(a "white rabbit" in an well known paper of Chow,1978)

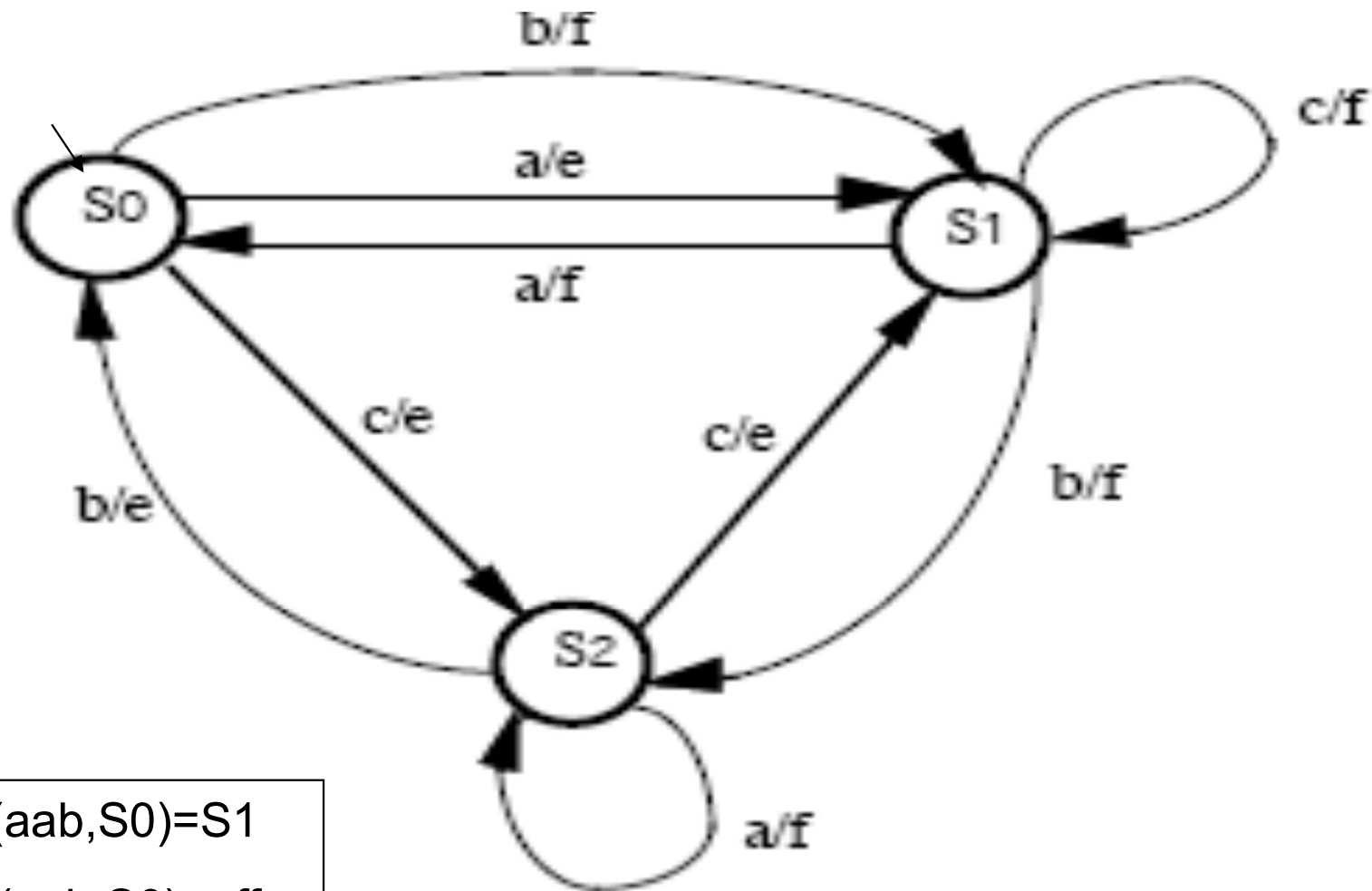
- User Requirements:
 - Input consists of characters `*`, `/`, `a`.
 - Print only comments
 - A comment is an input sequence enclosed by `/*` on the left and `*/` on the right (it may contain other `/*` 's but not `*/` 's)
- DFSM
 - $X = \{*, /, a\}$
 - $Y = \{\text{ignore, empty, acc-bf, deacc-bf, print-bf, deacc\&print-bf}\}$
 - T and O, given by the following diagram:

Diagram for “C comments” DFSM



- 1- waiting for a comment to start
- 2- a possible comment start
- 3- accumulating the comment content
- 4- waiting for comment to end

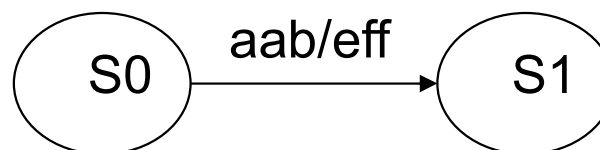
abc-DFSM, another example



$T(aab, S0) = S1$

$O(aab, S0) = \text{eff}$

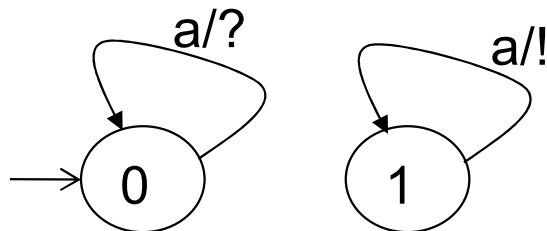
$O(aab, S1) = \text{fef}$



Complete, connected DFSM

- M is **completely** specified, if from each state of M there exists a transition and an output for each input symbol in X.
- M is **strongly connected**, if for each pair of states (q, p), there exists an input sequence which takes M from q to p.
- M is **connected**, if for each state q there exists an input sequence which takes M from the initial state q_0 to q.

C comments DFSM is complete and strongly connected



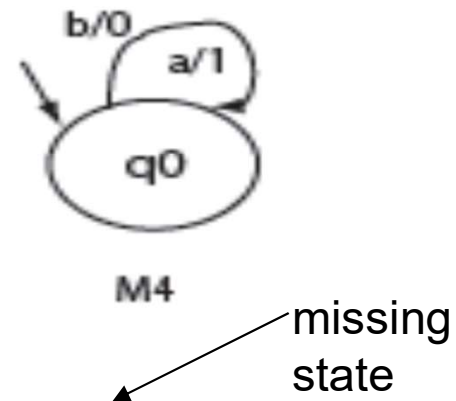
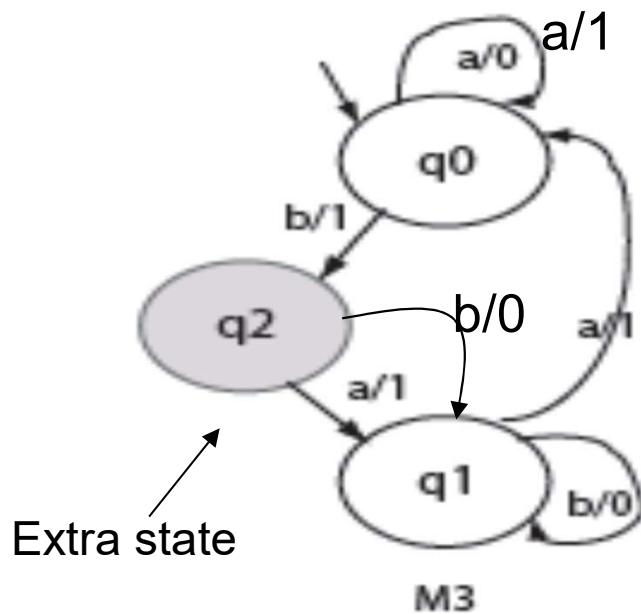
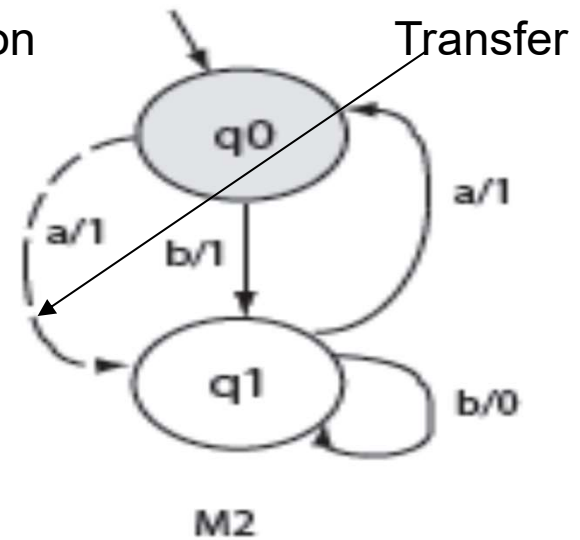
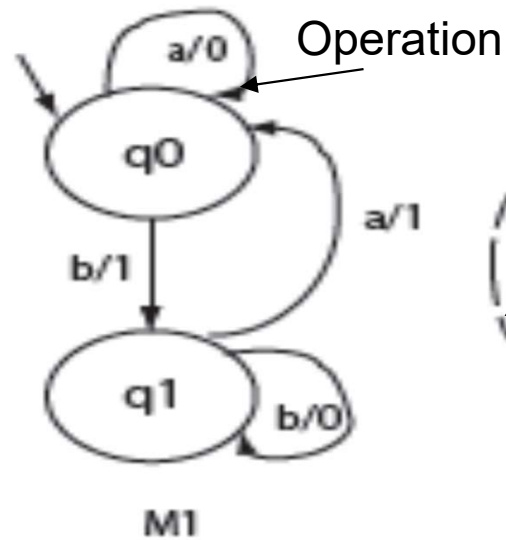
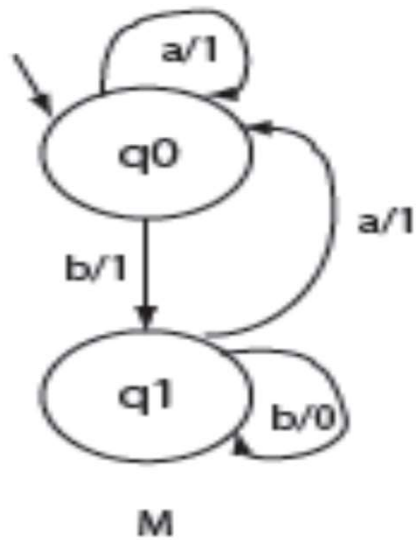
Completely specified,
not connected
(hence, not strongly connected)

(widely used) Fault models for FSM implementations

- *Operation error*
- *Transfer error*
- *Extra state error*
- *Missing state error:*

Example: FSM fault models

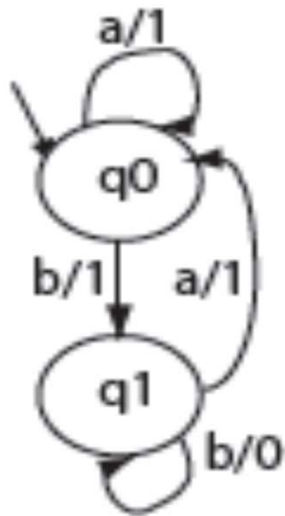
Correct
design



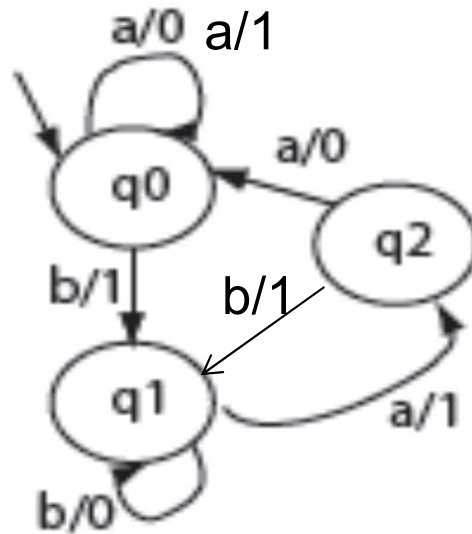
Equivalence

- Given a set V of input sequences, two states q and p are **V-equivalent** (written as “ $q \equiv(V) p$ ”), if q and p respond with identical output sequences to each input sequence in V .
 $\forall x(x \in V \Rightarrow O(x,q)=O(x,p))$
- Note. q and p are **distinguishable by V** if not equivalent on V
- Two states q and p are **equivalent** (written as “ $q \equiv p$ ”), if they are V -equivalent for any set V .
 $\forall x(O(x,q)=O(x,p))$
- Two FSMs **S and I** are **equivalent** if their initial states S_0 and I_0 are equivalent.
- Two states q and p are **k-equivalent**, $k \geq 1$ if they are X^k equivalent

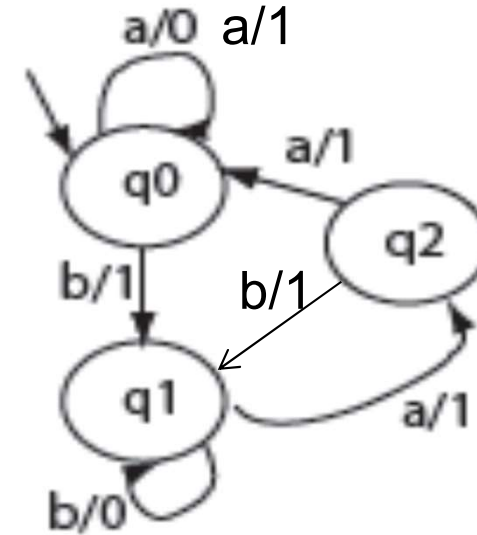
Example: extra state may or may not be an error



M



M1



M2

M1 and M2, extra state mutants of M

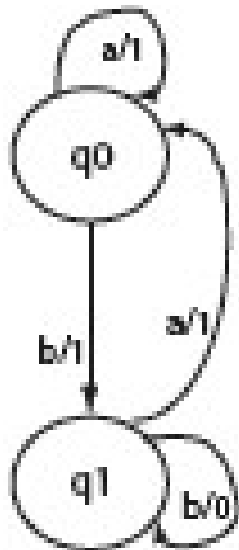
$M1 \not\equiv M$ (non equivalent FSMs),

$OM(abaa, q0) \neq OM1(abaa, q0)$

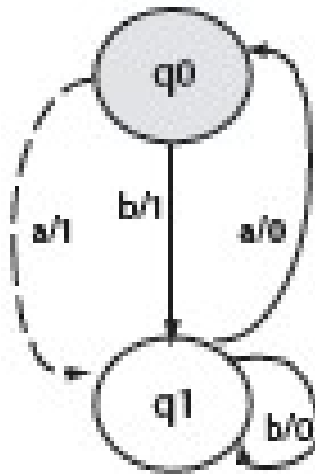
$M2 \equiv M$ (they are equivalent) (How to prove?)

Mutants of a given FSM

- A mutant of a FSM specification is an FSM obtained by introducing one or more errors zero or more times: the errors introduced belong to a given fault model

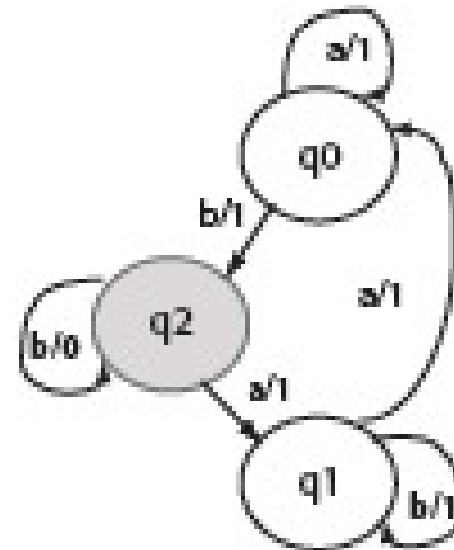


FSM M



Mutant M1

Operation and transfer
errors



Mutant M2

Operation and extra state
errors

How difficult is to find an error?

Take “C comments” FSM and consider a “transfer error mutant” replacing

$T(2,/)=2$ by $T(2,/)=1$

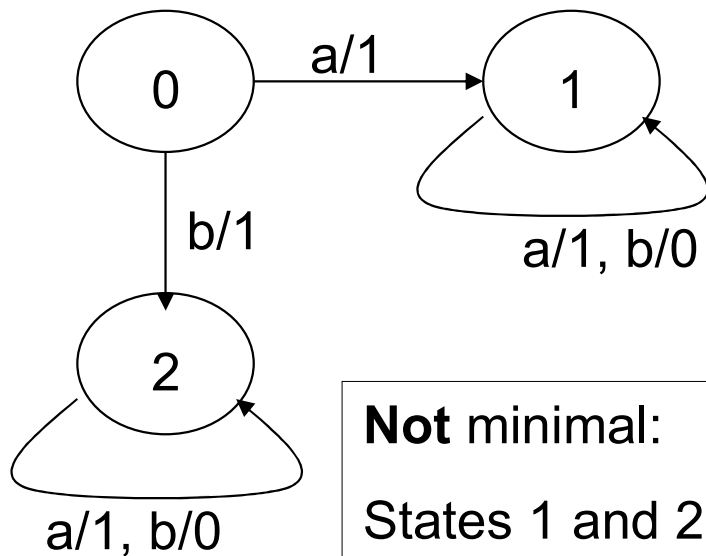
- Validation fails:
- `/*a*/`
contains a comment but it is not printed!
- Remark. The “error” is present in [Chow78]. Only 5% of my students catch the inadequacy!

How difficult is to prove a two FSM's equivalence?

- Prove that $M_2 \equiv M$, for the extra state mutant M_2
 - Show by induction on the length $n \geq 0$ of the input x :
for all n (for all x ($|x|=n$ implies
 $O(q_0, x) = O_2(q_0, x) = O_2(q_2, x)$ and
 $O(q_1, x) = O_2(q_1, x)$
))
- See why the same proof fails considering the extra state mutant M_1
- Remarks on proof ?

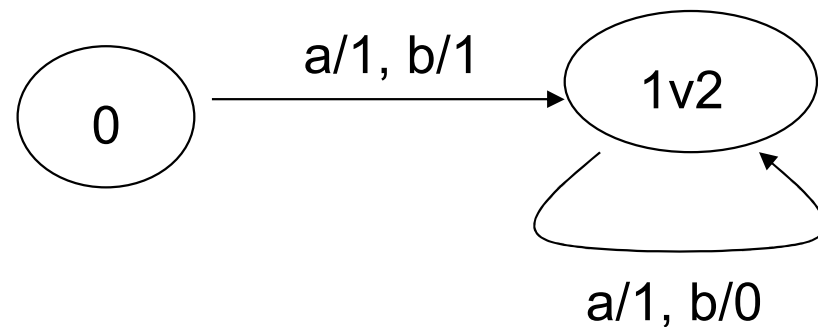
Minimal FSM

- An FSM M is **minimal** if the number of states in M is less than or equal to the number of states for any machine M' which is equivalent to M .



Not minimal:

States 1 and 2
are equivalent,
might be merged



Minimal:

(how to prove?)

Note: States 0 and 1v2

are distinguishable ($O(b,0)=1$, $O(b,1)=0$)

If one state, then $O(bb,q0)=11$ or 00)

Characterization set

- Most methods for generating tests from finite state machines make use of an important set known as the **characterization set**.
- Let $M = (X, Y, Q, q_0, T, O)$ be an FSM that is **minimal and complete**.
- A characterization set of M , denoted as W , is a finite set of input sequences that **distinguish the behavior of any pair of states** in M .

$$\forall p, q (p \neq q \Rightarrow \exists w \in W O(w, p) \neq O(w, q))$$

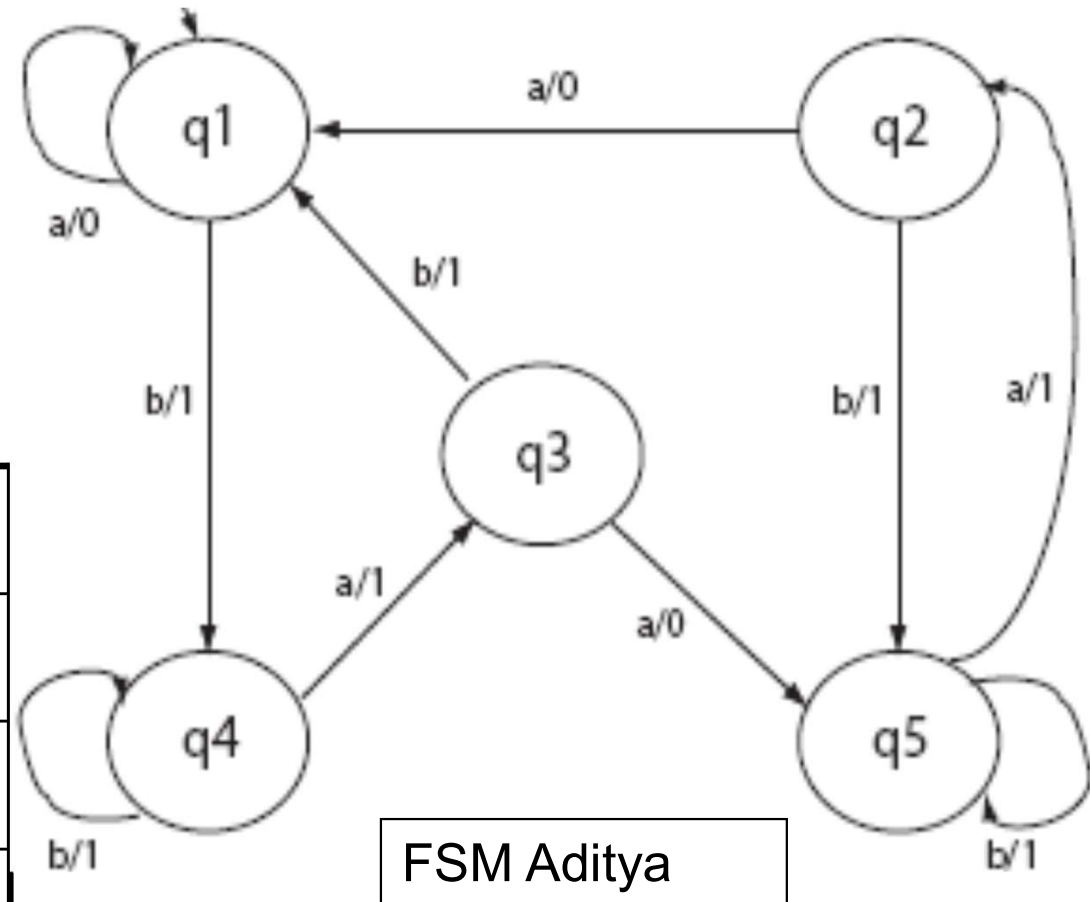
or

$$\forall p, q (p \neq q \Rightarrow p \not\equiv_W q)$$

Example: characterization set

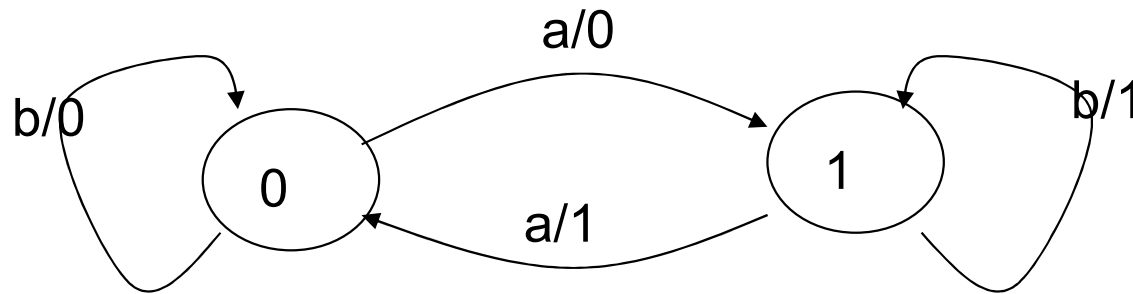
$W = \{a, aa, aaa, baaa\}$

	1	2	3	4	5
1	#	baaa	aa	a	a
2	#	#	aa	a	a
3	#	#	#	a	a
4	#	#	#	#	aaa
5	#	#	#	#	#



Note. All pairs are equivalent for shorter strings than those from W .
 $(1 \equiv (X^{<=3})2 \text{ a.s.o.})$

Might be more than one characterization set



$W1=\{a\}$

$W2=\{b\}$

Constructing a characterization set

The algorithm to construct a characterization set for an FSM M consists of two main steps.

1. The first step is to construct a sequence of k -equivalence partitions $P_1, P_2, \dots, P_k, \dots, P_m$, where $m \geq 1$
 - This iterative step converges in at most n steps where n is the number of states in M .
2. The W - procedure: in the second step these k -equivalence partitions are traversed, in reverse order, to obtain the distinguishing sequences for every pair of states.

k -equivalence partition

- Given an FSM $M = (X, Y, Q, q_1, O)$, a *k -equivalence partition* of Q , denoted by P_k , is a collection of n finite sets of states denoted as $Q_k^1, Q_k^2, \dots, Q_k^n$ such that
 - Union of Q_k^j is Q
 - States in Q_k^j are k -equivalent,
 - If $q \in Q_k^i, p \in Q_k^j$ and $i \neq j$, then q and p are k -distinguishable

Construction of the k -equivalence partitions

- We illustrate using the FSM Aditya
- Computing P1 (the 1-equivalence partition)
 1. write the transition and output functions for this FSM in a tabular form
 2. regroup the states that are identical in their Output entries
 3. Construct P1 table

Current state	Output		Next state	
	a	b	a	b
q ₁	0	1	q ₁	q ₄
q ₂	0	1	q ₁	q ₅
q ₃	0	1	q ₅	q ₁
q ₄	1	1	q ₃	q ₄
q ₅	1	1	q ₂	q ₅

1. T and O of
FSM, in tabular
form

2. regroup

P1 has

Q₁₁={q₁,q₂,q₃}

Q₁₂={q₄,q₅}

P1 table

3. Construct P1 table

- copy the “Next State” sub-table
- rename each Next State entry by appending a second subscript which indicates the group to which that state belongs.

Σ	Current state	Next state	
		a	b
1	q_1	q_{11}	q_{42}
	q_2	q_{11}	q_{52}
	q_3	q_{52}	q_{11}
2	q_4	q_{31}	q_{42}
	q_5	q_{21}	q_{52}

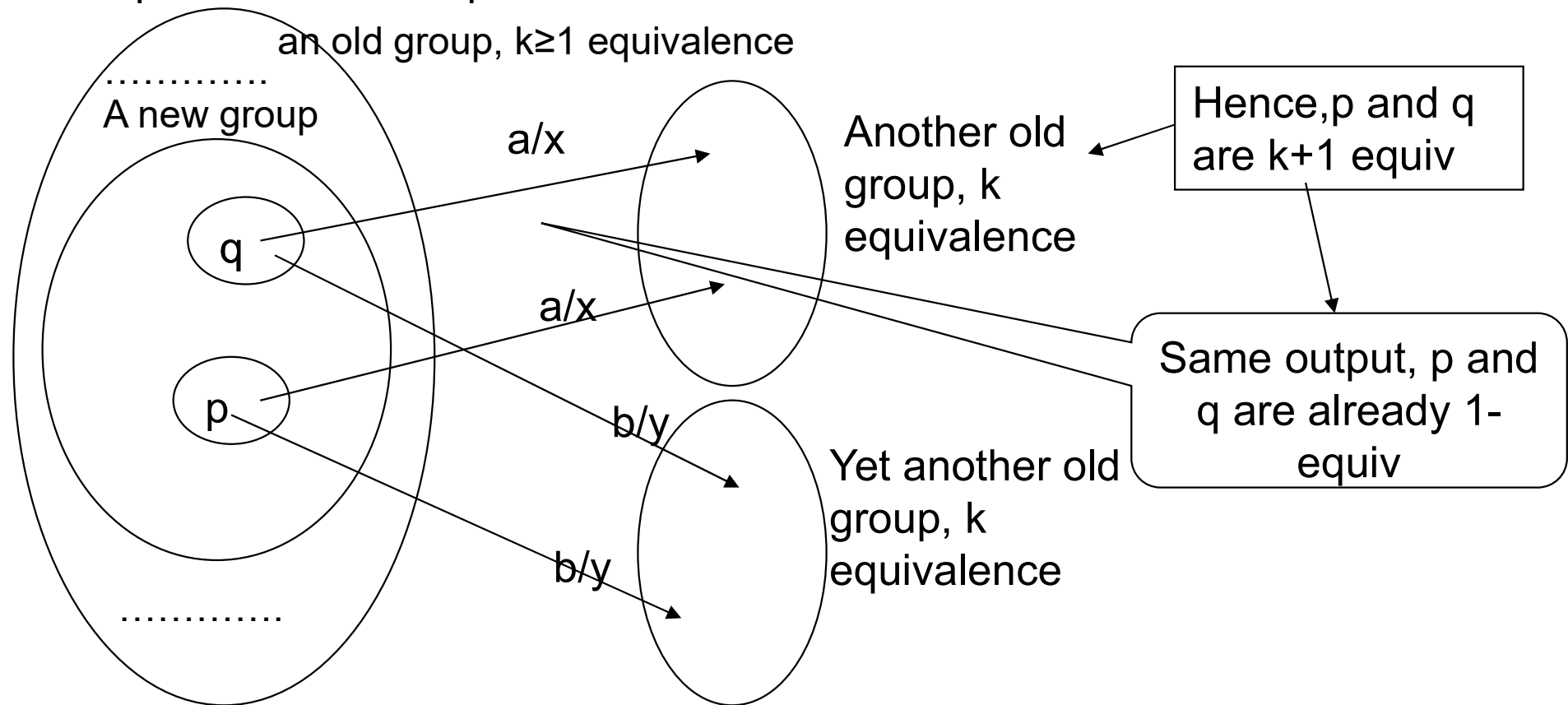
Important note.

States in the same group
are 1-equivalent

P_1 table.

Construct $P(k+1)$ table from P_k

- In every P_k group, regroup all rows with identical *second* subscripts in its row entries under the Next State column
 - states from the same new group, for every input, lead into states from the same old group (these are k -equivalent)
 - They are already 1-equivalent (were in the same group in P_1)
 - Hence, states from the same new group are $(k+1)$ equivalent
- relabel the groups
- update the subscripts associated with the next state entries.



P2 P3 tables

Σ	Current state	Next state	
		a	b
1	q_1	q_{11}	q_{43}
	q_2	q_{11}	q_{53}
2	q_3	q_{53}	q_{11}
3	q_4	q_{32}	q_{43}
	q_5	q_{21}	q_{53}

Σ	Current state	Next state	
		a	b
1	q_1	q_{11}	q_{43}
	q_2	q_{11}	q_{54}
2	q_3	q_{54}	q_{11}
3	q_4	q_{32}	q_{43}
4	q_5	q_{21}	q_{54}

P4 table

Σ	Current state	Next state	
		a	b
1	q_1	q_{11}	q_{44}
2	q_2	q_{11}	q_{55}
3	q_3	q_{55}	q_{11}
4	q_4	q_{33}	q_{44}
5	q_5	q_{22}	q_{55}

There are no distinct 4- equivalent states
There is, any pair of distinct states can be distinguished by an input of length 4

The W- procedure

{P1, ... Pn is the set of k-equiv partitions}

$W = \emptyset$;

forall $p \neq q$ do

if $\exists r(1 \leq r < n \wedge (p, q) \text{ equiv in } P_r \text{ but not equiv in } P_{r+1})$ then

choose such an r ; *// (p, q) r –equiv but not (r+1)-equiv*

$z = \varepsilon$; $p_1 = p$; $p_2 = q$;

for $m = r$ downto 1 do

choose x in X such that $G(p_1, x) \neq G(p_2, x)$ in P_m

$z = zx$;

$p_1 = T(x, p_1)$; $p_2 = T(x, p_2)$

end for

choose x such that $O(x, p_1) \neq O(x, p_2)$;

$z = zx$;

$W = W \cup \{z\}$;

else *{(p, q) not equiv in P1}* do

find x in X such that $O(x, q) \neq O(x, p)$;

$W = W \cup \{x\}$; *// x distinguishes q and p*

end if

end forall

$G(q, x)$ denotes the label of the group to which the machine moves

when excited using input x in state q .

For example, in the table for P_3

$G(q_2, b) = 4$

and

$G(q_5, a) = 1$.

Example: W procedure

For q_1 and q_2 we may find baaa, baba etc.

($r=3$; part if ...then of the W procedure)

For q_3 and q_4 we find a

(no r ; part if...else of the W procedure)

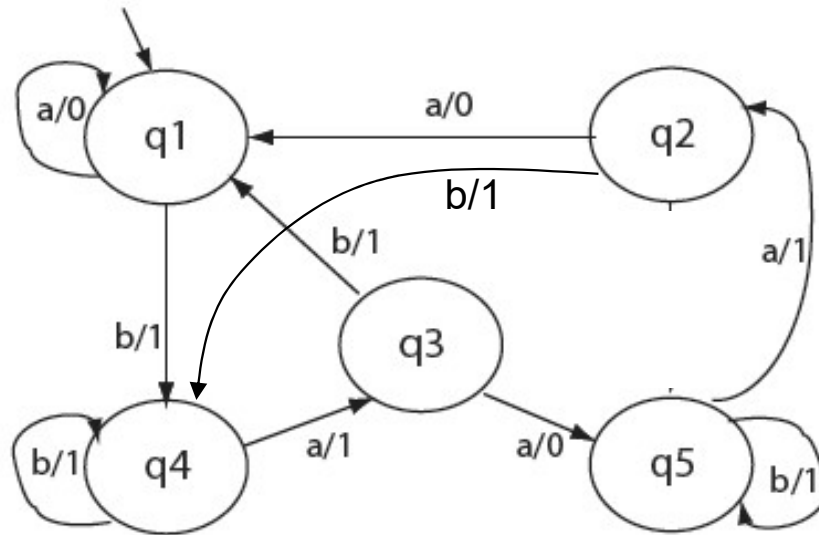
For q_4 and q_5 we may find aaa or aba.

($r=2$; part if ...then of the W procedure)

For q_2 and q_3 we may find aa or ba.

($r=1$; part if ...then of the W procedure)

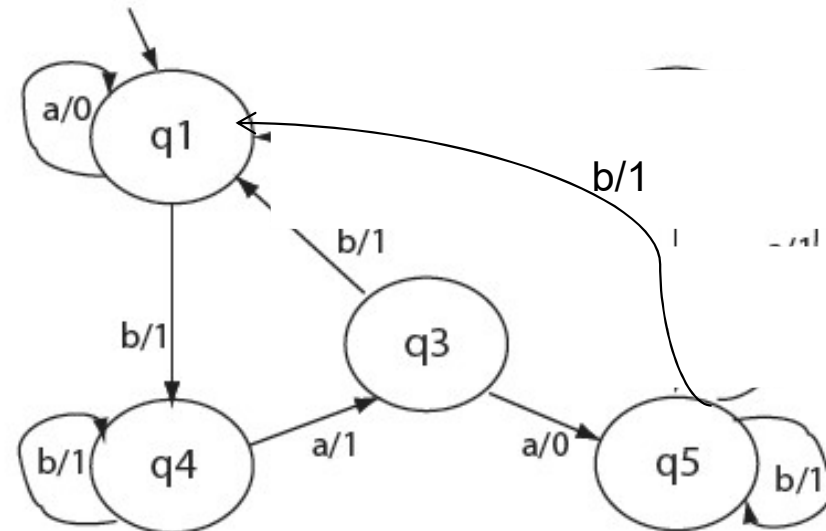
Exercise



1. Construct Pk equivalence tables
2. Find an equivalent minimal FSM.

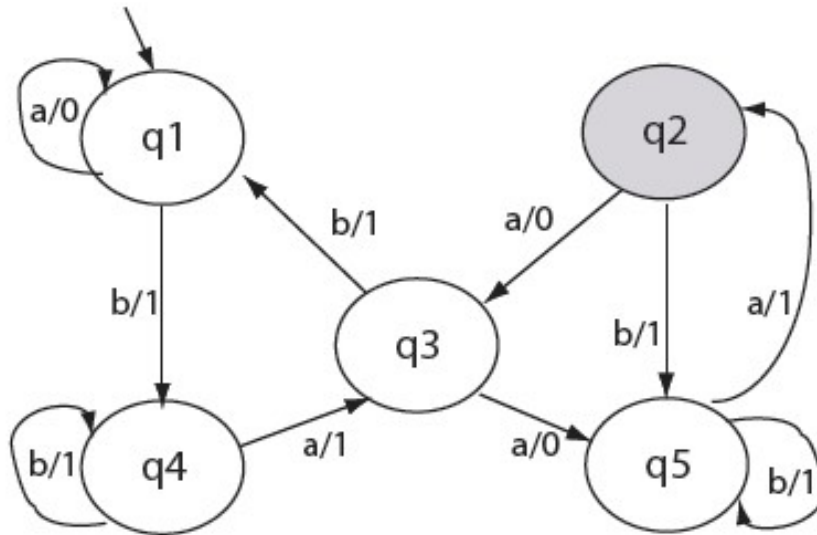
A.

1. We find $P3=P4$ and $q1 \equiv q2$
2. The minimal FSM is shown below.



Exercise

- Construct Pk equivalence classes.
- Find a characterization set

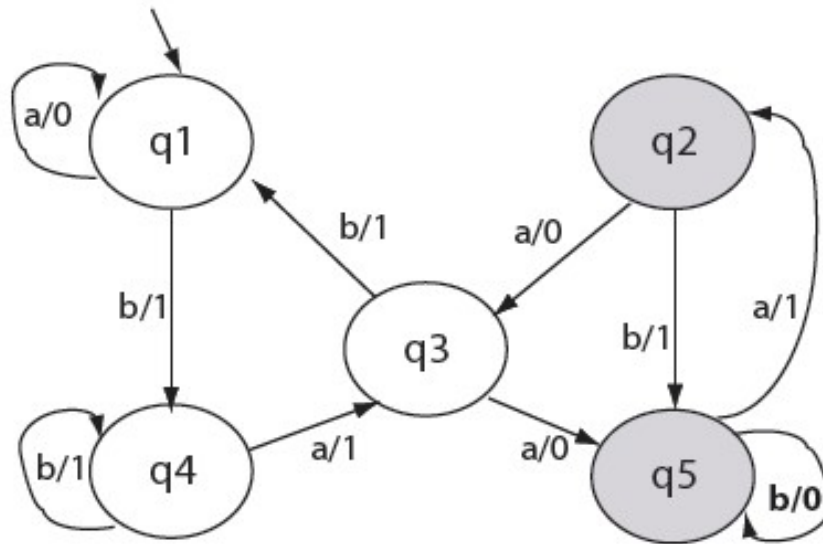


A.

- We find $P3=P4$, $|P3|=5$. The FSM is minimal
- $W=\{aaa\}$

Exercise

- Construct P_k equivalence classes.
- Find a characterization set



A.

- We find $P_2=P_3$, $|P_2|=5$. The FSM is minimal
- $W=\{a,ba,bb\}$

Identification sets

- Analogous to the characterization set for M , we associate an *identification* set with each state of M .
- An identification set for state q is denoted by Wq and has the following properties:
 - $Wq \subseteq W$
 - Wq is minimal with respect to
 $\forall p (p \neq q \Rightarrow \exists s \in Wq \ O(s,p) \neq O(s,q))$

EXAMPLE. Consider the machine FSM Aditya:
characterization set and its W shown in the table.
From the table we deduce:
 $W1=W2=\{baaa,aa,a\}$; $W3=\{a,aa\}$; $W4=W5=\{a,aaa\}$

- While the characterization sets are used in the W -method, the W_i sets are used in the W_p method.

The W-method (Chow)(1/?)

- The W-method is used for constructing a test set T from a given FSM specification S and knowing some information about the IUT I .
- $S \equiv (T) I \Rightarrow S \equiv I$
- The method makes some assumptions about the specification S and the IUT I .

Chow's W method assumptions

Hypothesis and assumptions:

- S is deterministic and minimal
- I is *assumed* to be deterministic
- S is **completely** specified and I is *assumed* to be as well
- all states in S are reachable and those for I are *assumed* to be as well
- The number of states in I is *assumed* to be bounded by an integer m , which may be larger than the number n of states in S

The W methods provides a test set $T=P \cdot Z$, where

- P is a transition cover of S
- $Z = (\{e\} \cup X \cup \dots X(m-n \text{ times})) \cdot W$
- W is a characterization set of S

State cover, transition cover

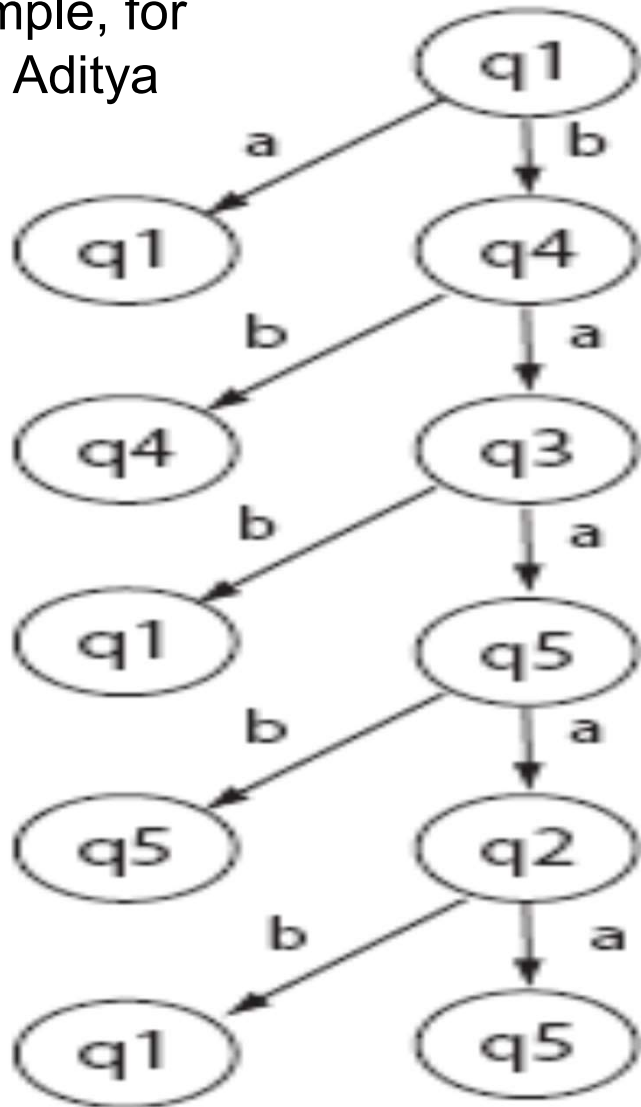
- Let Q be a set of input sequences. Q is a **state cover** set of S if for each state q of S , there is an input sequence x in Q such that $T(x, q_0) = q$.
 - For the initial state q_0 , we have $T(\epsilon, q_0) = q_0$. The empty input sequence belongs to Q .
 - Note: In many cases, one uses a state cover set that is closed under the operation of "selecting a prefix".
- Let P be a set of input sequences. P is a **transition cover** set of S if:
 - $Q \subseteq P$, Q a state cover
 - for each transition $p \xrightarrow{x/y} q$, there are sequences w and wx in P such that $T(w, q_0) = p$ and $T(x, p) = q$.
- The empty sequence is a member of P .
- By definition, each transition cover set P contains a subset which is also a state cover set.
- The set of all partial paths in the testing tree of S , as defined in [Chow 78], is a transition cover set. A procedure for the construction of this set is also given there.

Computation of the transition cover set (1/2)

- We can construct a transition cover set P using the “testing tree” of M .
- A testing tree for an FSM is constructed as follows.
 - State q_0 , the initial state, is the root of the testing tree. This is level 1 of the tree.
 - Suppose that the testing tree has been constructed until level k . The $(k + 1)$ th level is built as follows.
 - Select a node n at level k . If n appears at any level from 1 through k , then n is a leaf node and is not expanded any further. If n is not a leaf node then we expand it by adding a branch from node n to a new node m if $T(n, x) = m$ for $x \in X$. This branch is labeled as x . This step is repeated for all nodes at level k .
- Once a testing tree has been constructed, we obtain the transition cover set P by concatenating labels of all *partial paths* along the tree.

Testing tree

Example, for
FSM Aditya



$P =$

$\{\epsilon,$

$a, b,$

$bb, ba,$

$bab, baa,$

$baab, baaa,$

$baaab, baaaa\}$

Thus exciting an FSM with elements of P ensures that

- **all states are reached**, and
- **all transitions have been traversed** at least once.

Constructing Z

Suppose that the number of states estimated to be in the IUT is m and the number of states in the design specification is n , $m \geq n$. Given this information we compute Z as:

$$Z = X[m - n].W,$$

for $m = n$, i.e. when the number of states in the IUT is the same as that in the specification.

$$Z = X.W$$

For $m < n$ we still use

$$Z = X.W.$$

$$Z = (X^0.W) \cup (X.W) \cup (X^2.W) \dots \cup (X^{m-1-n}.W) \cup (X^{m-n}.W)$$

The W-method (Chow)(2/?)

Let be S having n states. W-method consists of the following sequence of steps.

- Step 1 Estimate the maximum number m of states in the IUT.
- Step 2 Construct the characterization set W for the given machine S .
- Step 3 Construct the testing tree for S and from it determine the transition cover set P .
- Step 4 Construct set $Z = X[m-n]W$
- Step 5 $P.Z$ is the desired test set.

Deriving a test set

The test set: $T = P.Z$.

Example (FSM Aditya).

- For $m=n=5$,

$Z = X^0.W = \{a, aa, aaa, baaa\}$

$T = P.Z =$

$\{\epsilon, a, b, bb, ba, bab, baa, baab, baaa, baaab, baaaa\} \cdot \{a, aa, aaa, baaa\} =$

$\{a, aa, aaa, aaaa, abaaa, ba, baa, baaa, baaaa, baaaaa, baaaaaa, baaaaaaa, baaaaabaaa, baaaba, baaabaa, baaabaaa, baaabbaaa, baaba, baabaa, baabaaa, baabbaaa, baba, babaa, babaaa, babbaaa, bba, bbba, bbbaa, bbbbaa\}$

For $m=6$,

$Z = W \cup X.W =$

$\{a, aa, aaa, baaa, aa, aaa, aaaa, abaaa, ba, baa, baaa, bbaaa\}$

Testing using the W-method

To test the given IUT M_i against its specification S , we do the following for each test input.

1. Find the expected response $S(t)$ to a given test input t . This is done by examining the specification. Alternately, if a tool is available, and the specification is executable, one could determine the expected response automatically.

2. Obtain the response $I(t)$ of the IUT, when excited with t in the initial state.

3. If $S(t) = I(t)$ then no flaw has been detected so far in the IUT.

$S(t) \neq I(t)$ implies an error in the design or the IUT under test

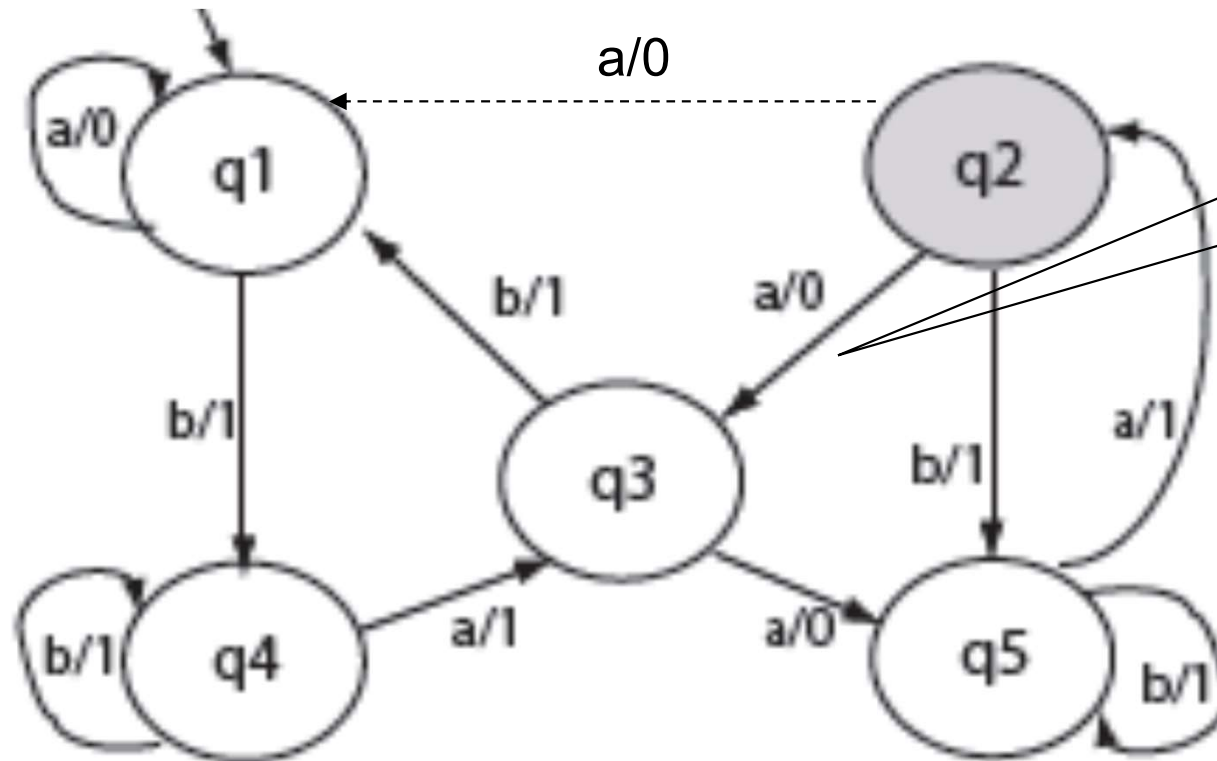
Example, testing with W- method

- S is Aditya.
- I is below, one error

With $t = baaaaaa$ we have $S(t) = 1101000$ and $I(t) = 1101001$. Thus the input sequence *baaaaaa* has revealed the transfer error in IUT.

transfer error

See some C
implementation
here

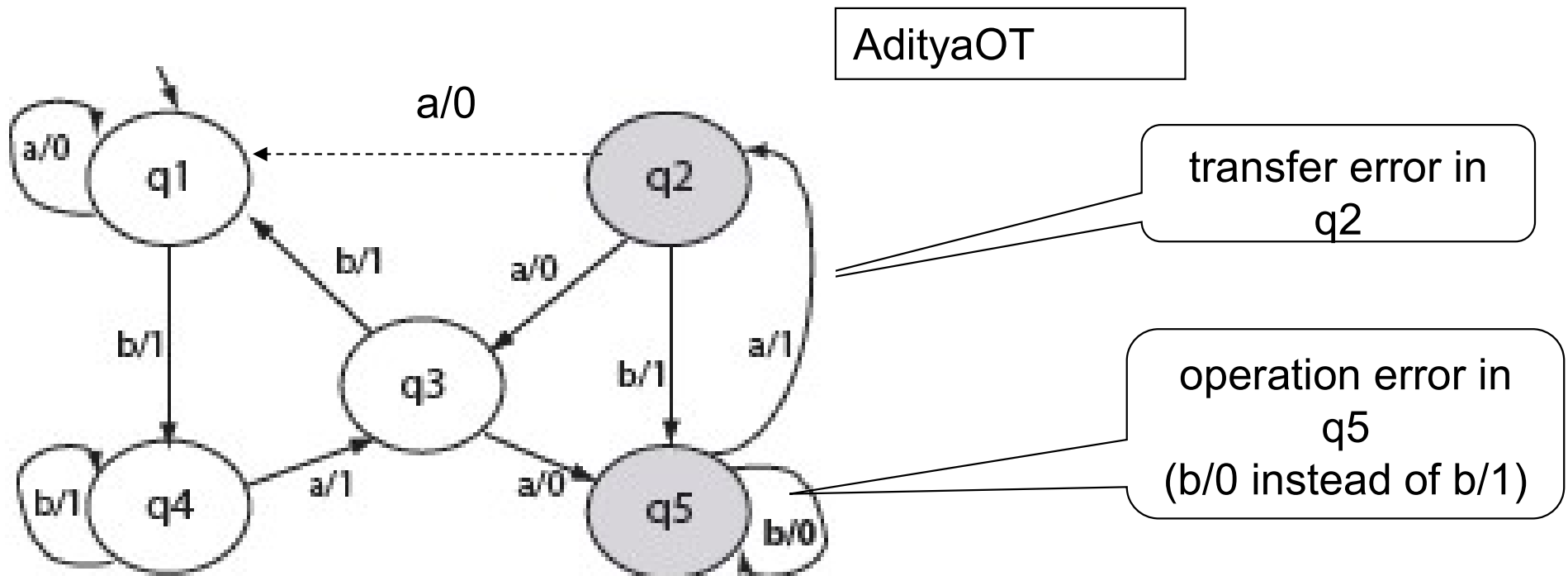


AdityaTr
transfer error in
q2

Example, testing with W- method

operation + transfer error

- S is Aditya.
- I is below, two errors.
With $t = baaba$ we have $S(t) = 11011$ and $I(t) = 11001$ and operation error is revealed
- With $t = baaaaa$, transfer error, already shown.

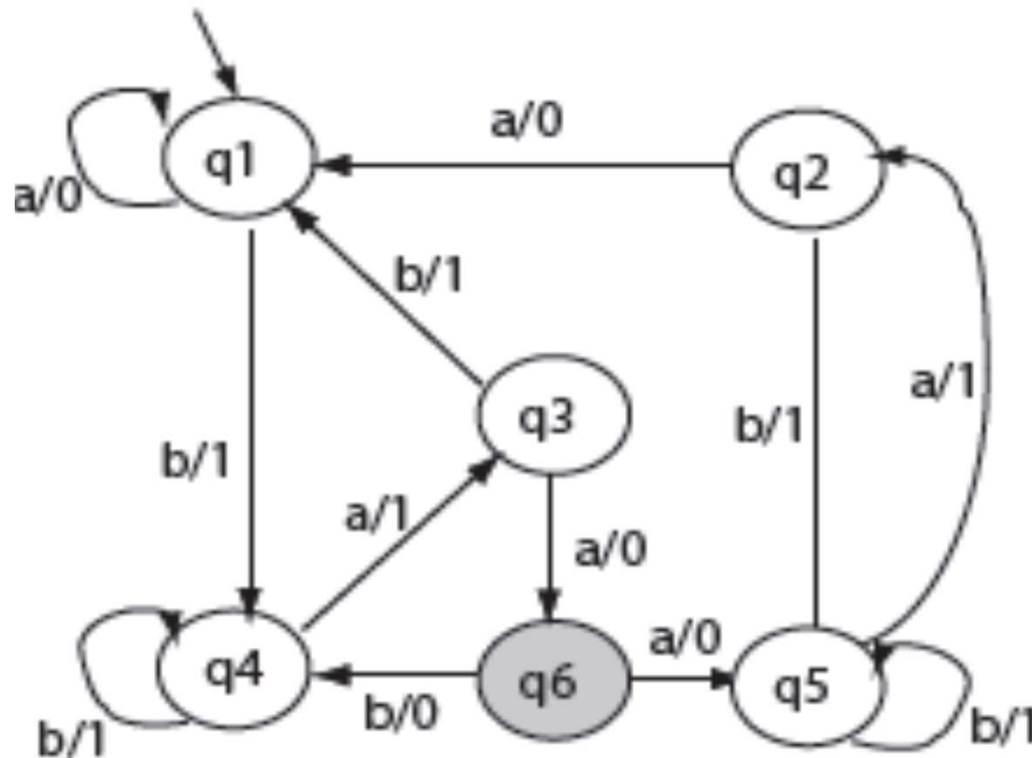


Example, testing with W- method

extra state error

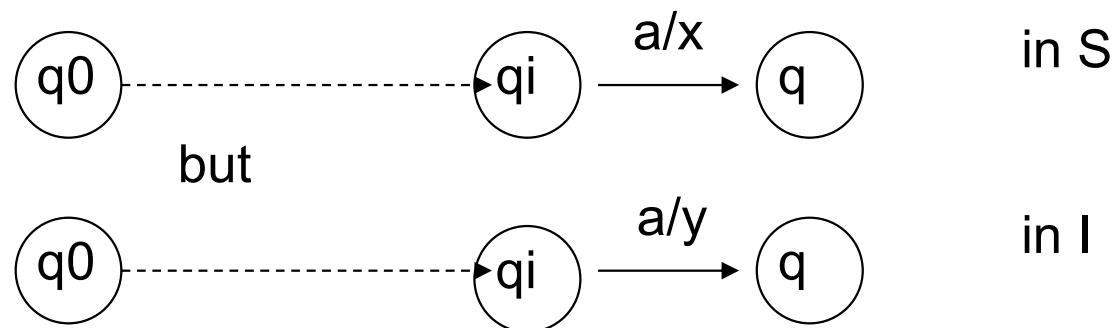
- S is Aditya.
- I is below, one error.

With $t = baaba$ we have $S(t) = 11011$ and $I(t) = 11001$ and extrastate error is revealed



The error detection process (1/2)

- $m=n$, $T=PW$
- examine carefully how the test sequences generated by the W-method detect operation and transfer errors.
- Operation error. Let us suppose that:

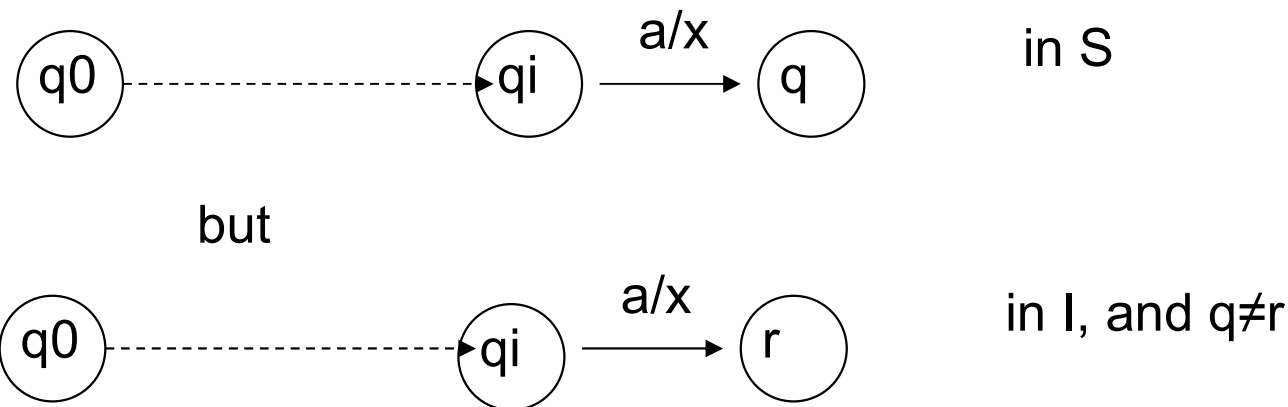


In P (transition cover) there exist p such that $q_0 \xrightarrow{p} q_i$

We have also pa in P , by def, thus operation error is detected when testing paw for a w in W .

The error detection process (2/2)

- $m=n$, $T=PW$
- Transfer error. Let us suppose that:



There is $w \in W$ such that $O(w, q) \neq O(w, r)$. Then $paw \in PW$ and the error is revealed

Wp- method

- The assumptions about S and I made for this method are similar to those made for the W -method.
- The main advantage of the Wp -method, over the W -method, is to reduce the length of the test suite.
- Instead of using the set W to check each reached state S_i , only a subset of this set can be used in certain cases.
 - This subset W_i depends on the reached state S_i , and is called an **identification set** for S_i .
 - The union of identification sets W_i for all states S_i is a characterization set .

Phases of Wp-method

- The Wp-method consists of two phases which have the following purposes:
- Phase 1: This phase checks that all the states defined by the specification are identifiable in the implementation, and also checks, for each state lk , that it can be identified by the smaller set Wk . At the same time, the transitions leading from the initial state to these states are checked for correct output and state transfer.
- Phase 2: This phase checks the implementation for all the transitions defined by the specification, which were not checked during the first phase.

Wp- method, case $m=n$

Wp-method proceeds as follows:

- A transition cover set P is determined which includes a state cover set Q .
- For each state S_i of S , an identification set W_i is determined and W is defined as the union of the W_i .
- *Phase 1:* The test sequences of Phase 1 are $Q.W$
Each state S_i of the specification is checked in the implementation with the W set. If the test is successful, we have $S \equiv (Q.W) I$.
- *Phase 2:* consist of the sequences of P that are not contained in Q , concatenated with the corresponding W_i

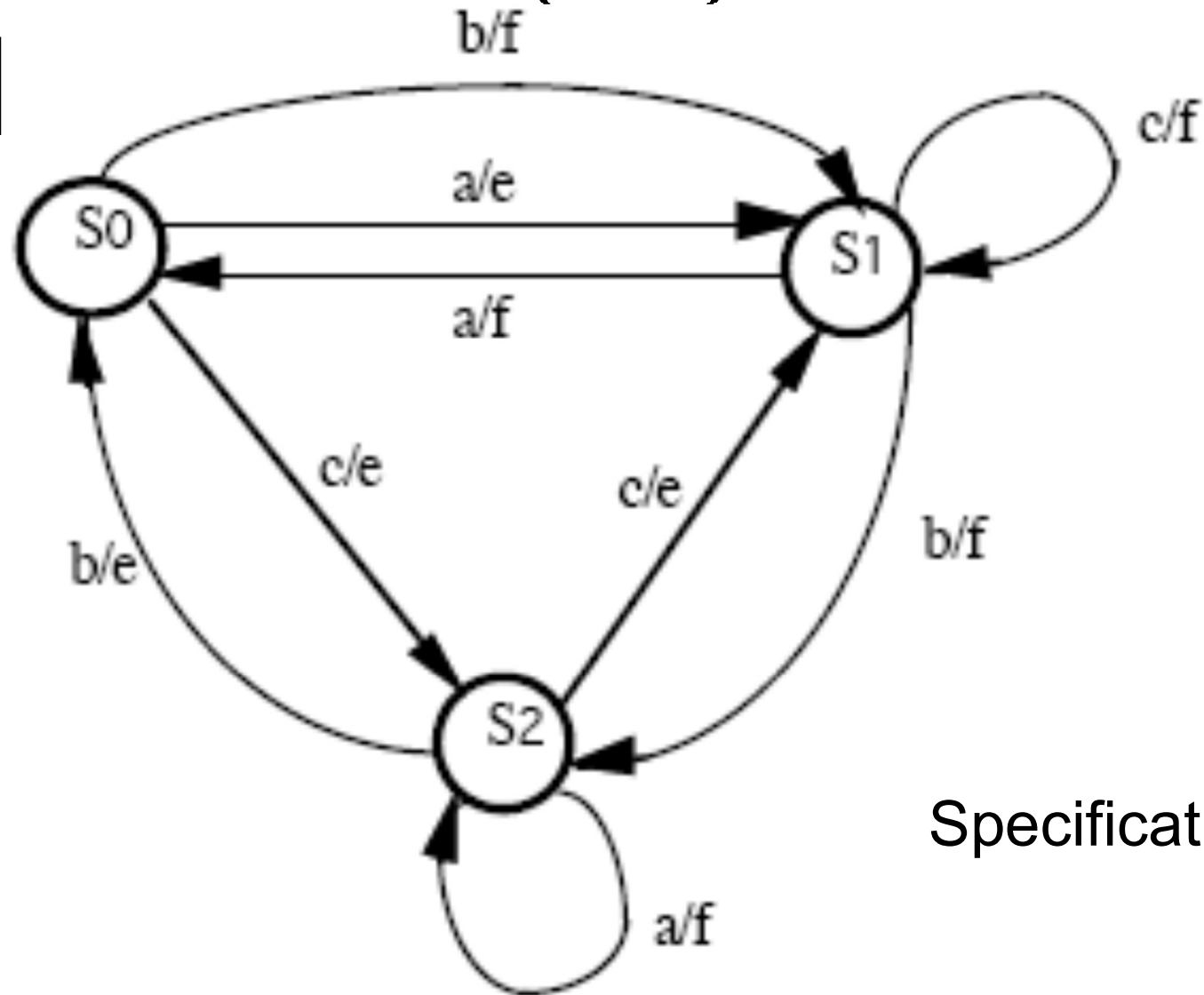
$$R \otimes W = \mathbf{Union} \text{ (forall } p \text{ in } R) \{p\} W_j,$$

where $R = P - Q$ and W_j is the identification set of S_j in W , and S_j is reached by p from the initial state.

- *Note1: If the implementation I passes the tests of both phases, it is equivalent to the specification S .*

Example Wp- method, case $m=n$ (1/?)

Fuji



Specification S

Example Wp- method, case $m=n$ (2/?)

- characterization set: $W = \{ a, b \}$.
- In fact we have
 - $W_0 = \{a\}$, distinguishes the state S_0 from all other states,
 - $W_1 = \{a, b\}$,
 - $W_2 = \{b\}$,
- $W = \{\{a\}, \{a, b\}, \{b\}\}$.
- $Q = \{\epsilon, b, c\}$ is a state cover set for S .
- $P = \{\epsilon, a, b, b.c, b.a, b.b, c, c.a, c.c, c.b\}$ is a transition cover set for S , which includes Q .
- $R = P - Q = \{ a, b.c, b.a, b.b, c.a, c.c, c.b \}$.

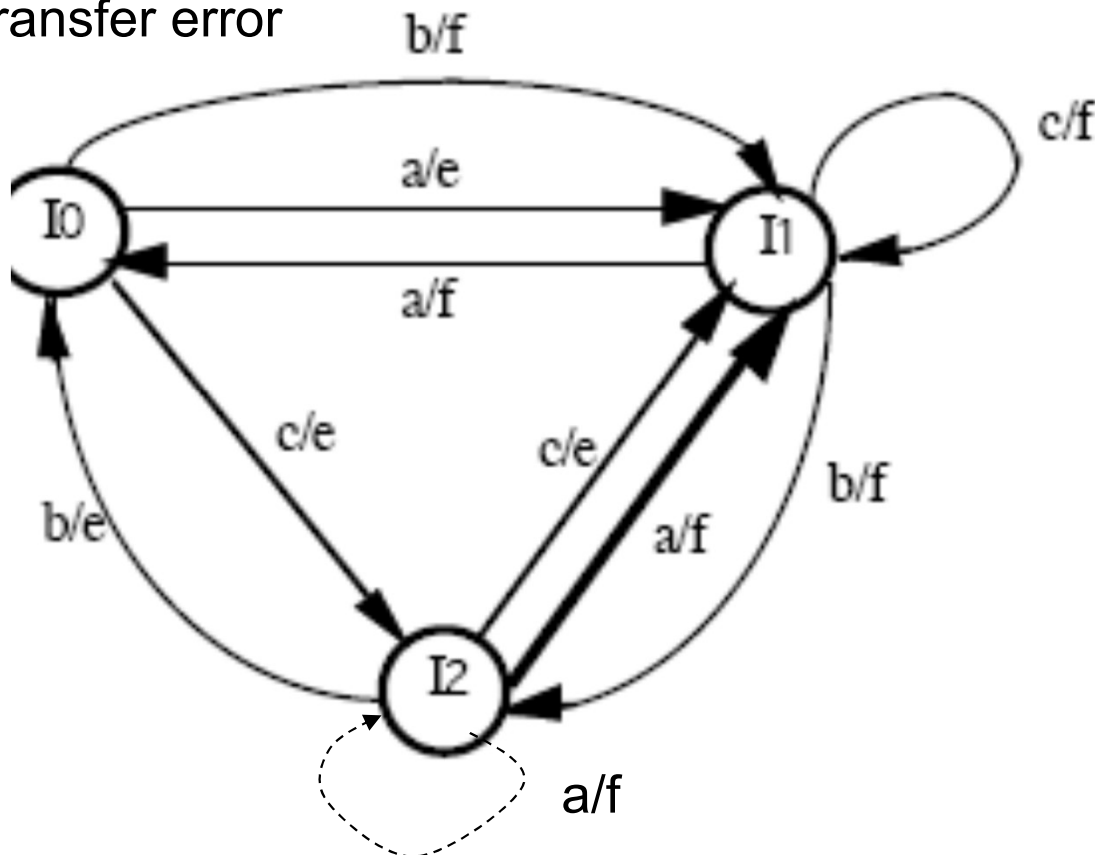
Example Wp- method, case $m=n$ (3/?)

- The Wp-method yields the following test sequences:
 - Phase 1:
 $T1=Q.W = \{a, b, b.a, b.b, c.a, c.b\}$
 - Phase 2 :
 $T2=R \otimes W = \{a.a, a.b, b.c.a, b.c.b, b.a.a, b.b.b, c.a.b, c.c.a, c.c.b, c.b.a\}$
- $R = P - Q = \{a, b.c, b.a, b.b, c.a, c.c, c.b\}$.
 $T(a,S0) = S1 \rightarrow W1$
 $\{a\}.W1 = \{a\}.\{a,b\} = \{aa, ab\}$
 $T(bc,S0)=S1 \rightarrow W1$
 $\{bc\}.\{a,b\} = \{bca, bcb\}$
 $T(ba,S0) = S0 \rightarrow W0$
 $\{ba\}.\{a\}=\{baa\} \dots\dots$
- Note: for the same sets P and W, the W-method generates the following additional test sequences:
b.a.b, b.b.a, c.a.a, c.b.b.

Example Wp- method, transfer error(1/?)

The faulty implementation I contains a transfer fault (with respect to Fuji)

Fuji-transfer error



Phase 1 (QW):
No faults detected

Phase 2 ($R \otimes W$):
Input **cab**
Output of **f**
instead of **e**,
Error detected.

Example Wp- method, transfer error(2/?)

- Test suites can be further optimized.
 - the tests of Phase 1 are included in the tests of Phase 2 (e.g. a is included in aa, ba in baa etc.)
 - Therefore, only the tests of Phase 2 need to be executed.

Exercise Wp- method, 1

- For the specification Fuji, $W1' = \{c\}$ can be chosen for state S1 as an identification set.
 - Construct a Wp- test set using $W' = \{\{a\}, \{c\}, \{b\}\}$
 - Optimize the constructed test set
- R.
 - Phase 1: The test sequences for this phase are:
 $Q.W' = \{a, b, c, b.a, b.b, b.c, c.a, c.b, c.c\}$
 - Phase 2 : The test sequences for this phase are:
 $R \otimes W' = \{a.c, b.c.c, b.a.a, b.b.b, \mathbf{c.a.b}, c.c.c, c.b.a\}$
 - Phase 1 may be skipped.
 - Note: **cab** is a test sequence in Phase 2, the implementation Fuji-transfer fails again the test

Wp-method ($m > n$)

For implementations with additional states ($m > n$),

- Phase 1 uses **QZ** ($Z = X[m-n].W$) instead of QW

- Phase 2 uses

$$\mathbf{RX}[m-n] \otimes \mathbf{W} =$$

Union (for all r in R) $\{r\} Z_i$

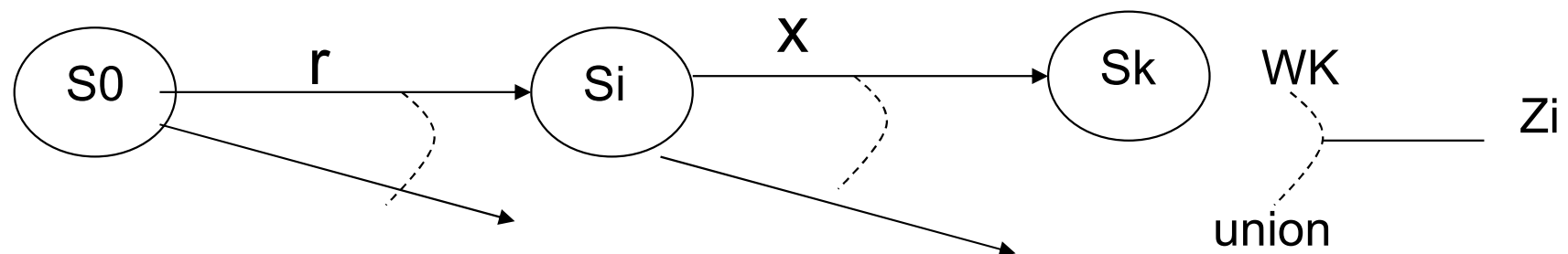
where

W_k is the identification set of S_k

S_k is reached from S_i by x

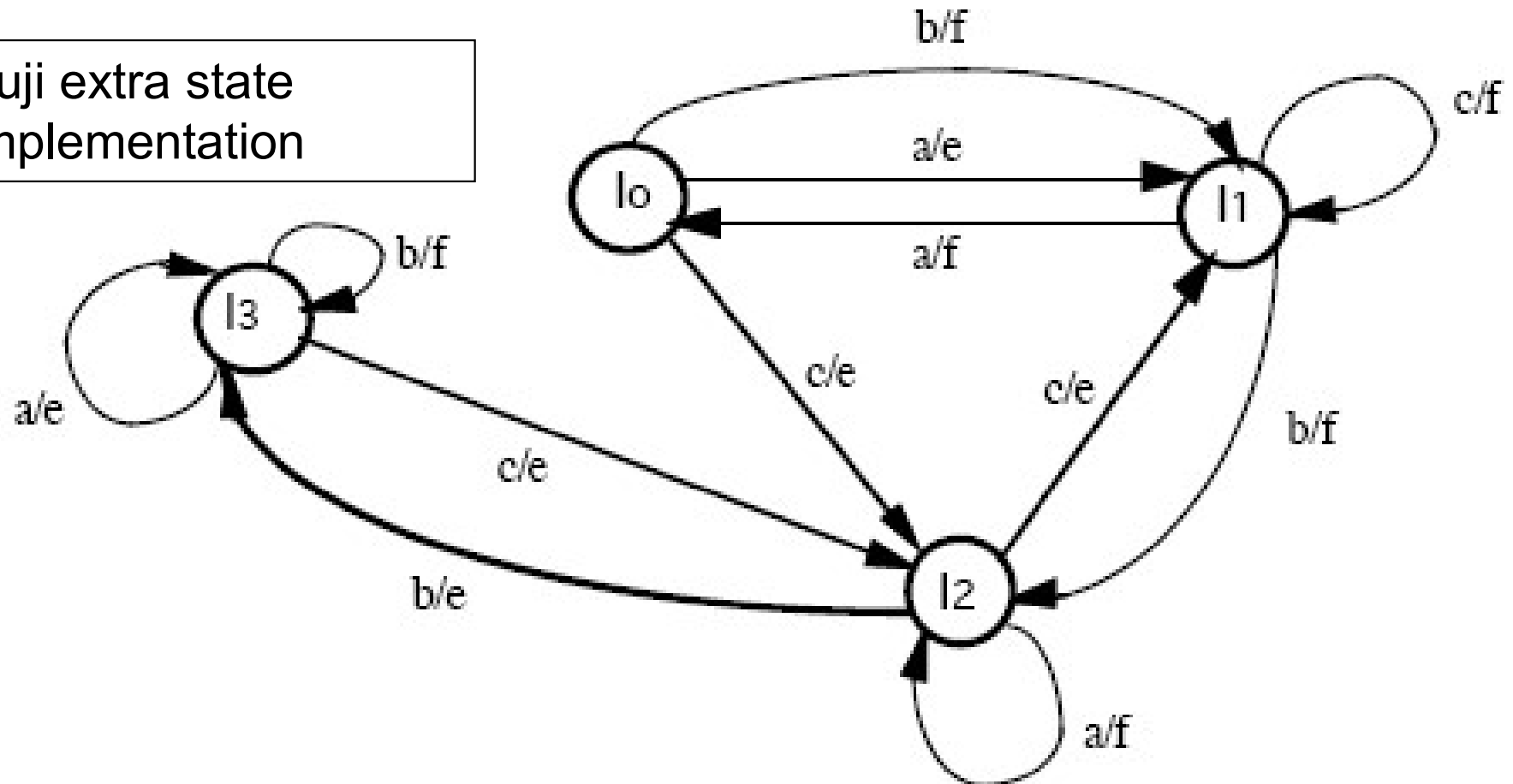
S_i is reached from S_0 by r

$Z_i = [\mathbf{Union} \text{ (for all } x \text{ in } X[m-n]) \{x\} W_k]$,



Example Wp- method, case $m > n$ (1/?)

Fuji extra state implementation



The extra state l_3 has the same outputs as l_0 for all inputs, but differs in the next states.

Example Wp- method, case $m > n$ (2/?)

- If we take **$m=3$** and apply the Wp-method, the faulty implementation **passes** the test.
- The W method with **$m=3$, $W=\{a,b\}$** does **not detect** the fault either.
 - Since I_3 is W-equivalent to I_0 , the characterization set W can not distinguish I_3 from I_0 .

Example Wp- method, case $m > n$ (3/?)

Let us take

- $m=4$
- $W = \{\{a\}, \{a, b\}, \{b\}\}$.
- $Q = \{\varepsilon, b, c\}$, state cover set for Fuji.
- $P = \{\varepsilon, a, b, b.c, b.a, b.b, c, c.a, c.c, c.b\}$,
transition cover
- Then the Wp-method yields test:

Example Wp- method, case $m > n$ (4/?)

- Phase 1: The test sequences for this phase are:

$$T1=Q.W = W \cup \{b\}.W \cup \{c\}.W$$

$$= \{a, b, b.a, b.b, c.a, c.b\} \text{ (already computed)}$$

$$Q.X.W = \{a\}.W \cup \{b\}.W \cup \{c\}.W \cup \{b.a\}.W \cup \{b.b\}.W \\ \cup \{b.c\}.W \cup \{c.a\}.W \cup \{c.b\}.W \cup \{c.c\}.W$$

$$= \{a.a, a.b, b.a, b.b, c.a, c.b,$$

$$b.a.a, b.a.b, b.b.a, b.b.b, b.c.a, b.c.b,$$

$$c.a.a, c.a.b, c.b.a, c.b.b, c.c.a, c.c.b\}$$

Example Wp- method, case $m > n$ (5/?)

- Phase 2 : The test sequences for this phase are:
 $R \otimes W =$
 $\{a.a, a.b, b.c.a, b.c.b, b.a.a, b.b.b, c.a.b, c.c.a, c.c.b,$
 $c.b.a \}$
(already computed)
 $R.X \otimes W = \{a.a\}.W0 \cup \{a.b\}.W2 \cup \{a.c\}.W1 \cup$
 $\{b.c.a\}.W0 \cup \{b.c.b\}.W2 \cup \{b.c.c\}.W1 \cup$
 $\{b.a.a\}.W1 \cup \{b.a.b\}.W1 \cup \{b.a.c\}.W2 \cup$
 $\{b.b.a\}.W2 \cup \{b.b.b\}.W0 \cup \{b.b.c\}.W1 \cup$
 $\{c.a.a\}.W2 \cup \{c.a.b\}.W0 \cup \{c.a.c\}.W1 \cup$
 $\{c.c.a\}.W0 \cup \{c.c.b\}.W2 \cup \{c.c.c\}.W1 \cup$
 $\{c.b.a\}.W1 \cup \{c.b.b\}.W1 \cup \{c.b.c\}.W2$

Example Wp- method, case $m > n$ (6/?)

- That is:

$R.X \otimes W = \{a.a.a, a.b.b, a.c.a, a.c.b, b.c.a.a, b.c.b.b, b.c.c.a, b.c.c.b, b.a.a.a, b.a.a.b, b.a.b.a, b.a.b.b, b.a.c.b, b.b.a.b, b.b.b.a, b.b.c.a, b.b.c.b, c.a.a.b, c.a.b.a, c.a.c.a, c.a.c.b, c.c.a.a, c.c.b.b, c.c.c.a, c.c.c.b, c.b.a.a, c.b.a.b, c.b.b.a, c.b.b.b, c.b.c.b\}$

- Phase 1. No faults are detected during this phase. Note. $\{c.b\}.W$ visits the extra state l3 in the implementation and checks the outputs for W.
- Phase 2.
 - $R \otimes W$, no error
 - $R.X \otimes W$, error on cbaa
- Optimizing:

only the tests $R.X \otimes W$ of Phase 2 need to be executed

- **Note.** Wp-method can detect faults including extra states if the bound m is chosen properly.

Cover criteria (1/2)

- *State cover:*
A test set T is considered adequate with respect to the state cover criterion for an FSM M if the execution of M against each element of T causes each state in M to be visited at least once.
- *Transition cover:*
A test set T is considered adequate with respect to the branch, or transition, cover criterion for an FSM M if the execution of M against each element of T causes each transition in M to be taken at least once.
- *Switch cover:*
A test set T is considered adequate with respect to the 1-switch cover criterion for an FSM M if the execution of M against each element of T causes each pair of transitions $(tr1, tr2)$ in M to be taken at least once, where for some input substring ab $tr1 : qi = T(qj, a)$ and $tr2 : qk = T(qi, b)$

Cover Criteri (2/2)

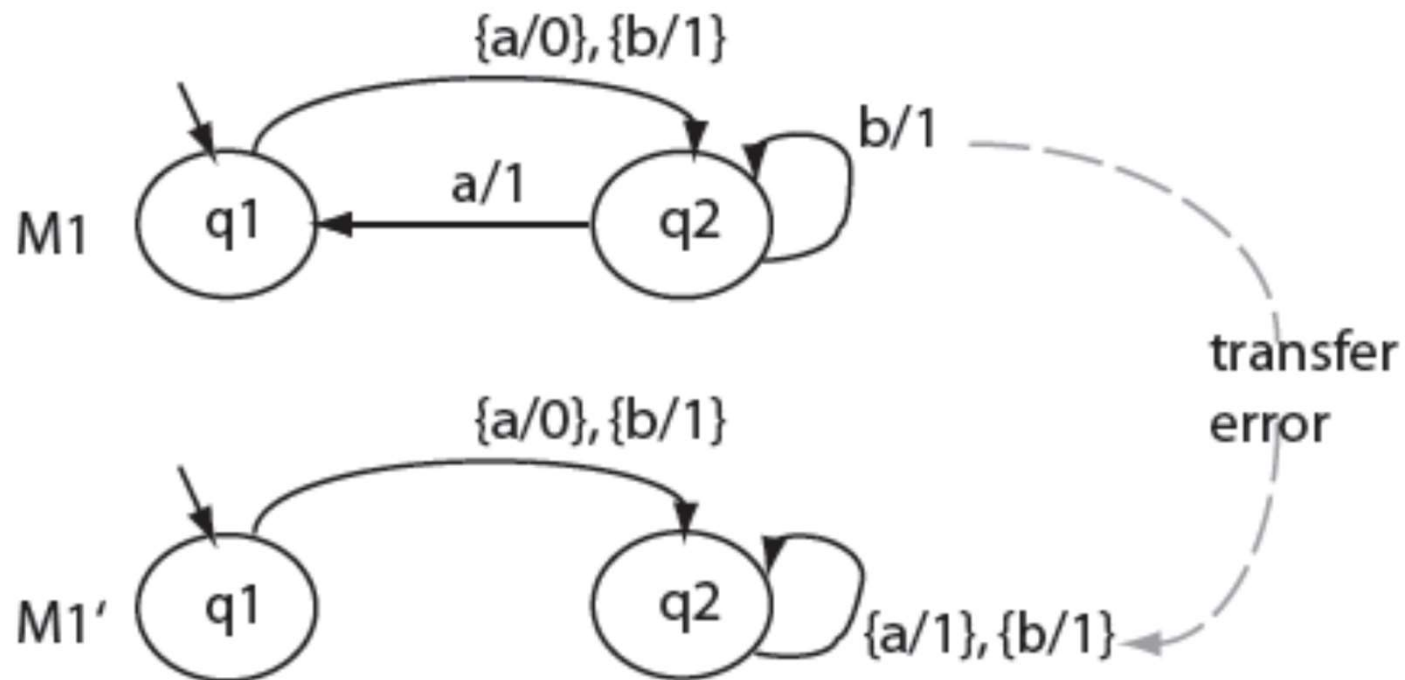
- Tests generated using the W- and the Wp methods guarantee the detection of all missing transitions, incorrect transitions, extra or missing states, and errors in the output associated with a transition given that the underlying assumptions hold.
- tests generated using these methods are more effective in detecting faults than the tests that are found adequate with respect to *state cover*, *branch cover*, *switch cover*

State cover, transition cover versus W

Consider the input sequence $t = abba$.

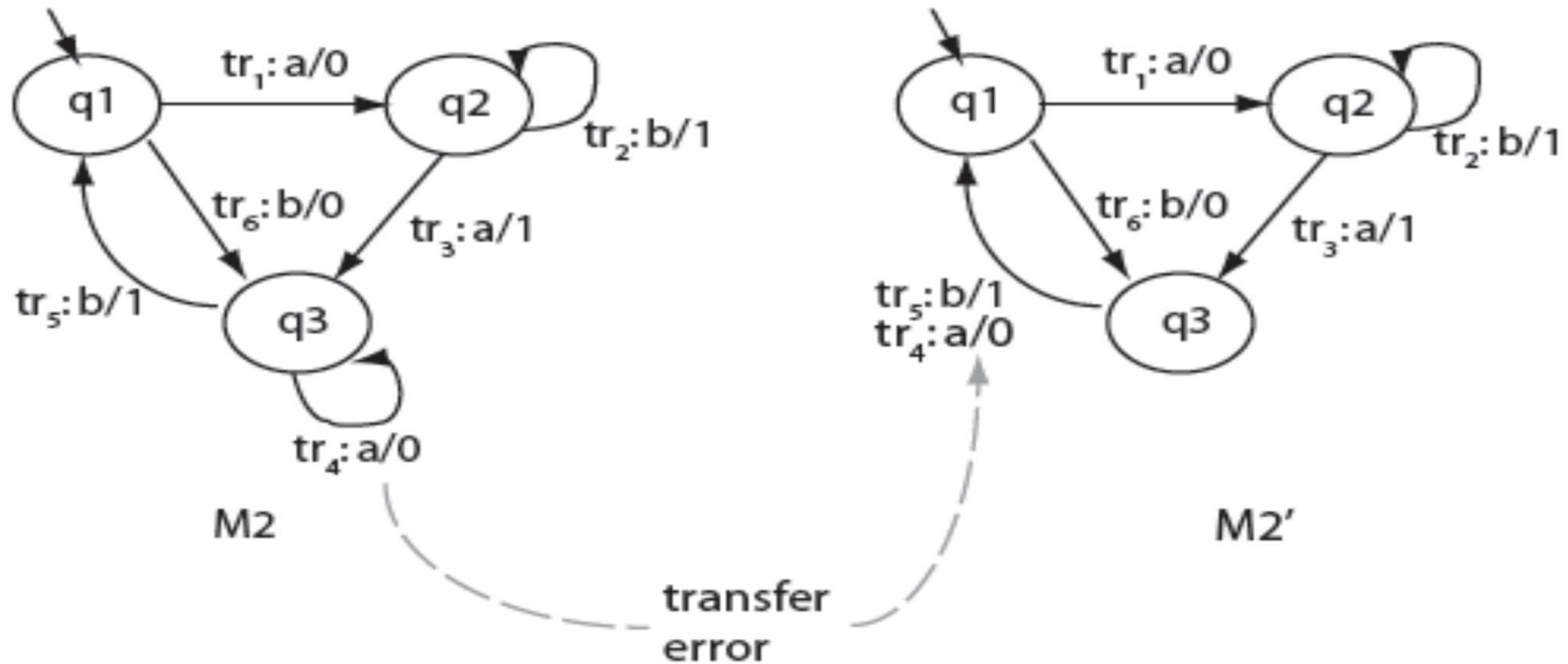
t covers all states and all transitions in $M1$ and hence is adequate with respect to the state coverage and transition coverage criteria.

However, it does not reveal the transfer error in $M1'$.



Switch criterion versus W (1/2)

- the following set of branch pairs must be exercised.
 $S = \{(tr1, tr2), (tr1, tr3), (tr2, tr2), (tr2, tr3), (tr3, tr4), (tr3, tr5), (tr4, tr4), (tr4, tr5), (tr5, tr6), (tr5, tr1), (tr6, tr4), (tr6, tr5)\}$



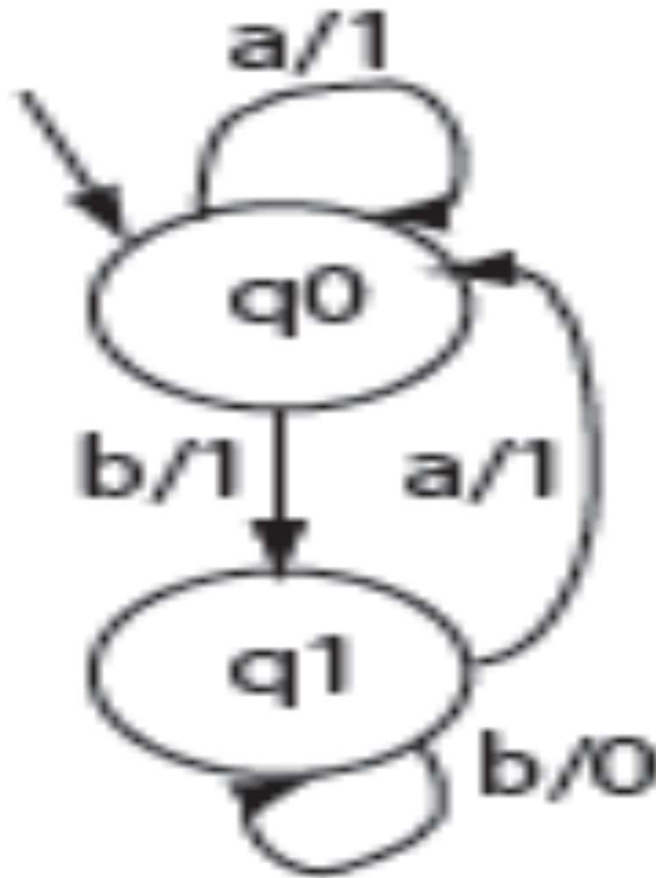
Switch criterion versus W (2/2)

- The following table lists a set of test sequences adequate with respect to the switch cover criterion but do not reveal the transfer error in state q_3

Test sequence (t)	$O_{M_2}(q_1, t)$ ($= O_{M_2'}(q_1, t)$)	Switches covered
<i>abb₁aa₂b</i>	0111001	$(tr_1, tr_2), (tr_2, tr_2), (tr_2, tr_3), (tr_3, tr_4),$ $(tr_4, tr_4), (tr_4, tr_5)$
<i>aaba</i>	0110	$(tr_1, tr_3), (tr_3, tr_5), (tr_5, tr_1)$
<i>aabb</i>	0110	$(tr_1, tr_3), (tr_3, tr_5), (tr_5, tr_6)$
<i>baab</i>	0001	$(tr_6, tr_4), (tr_4, tr_4), (tr_4, tr_5)$
<i>bb</i>	01	(tr_6, tr_5)

Exercises

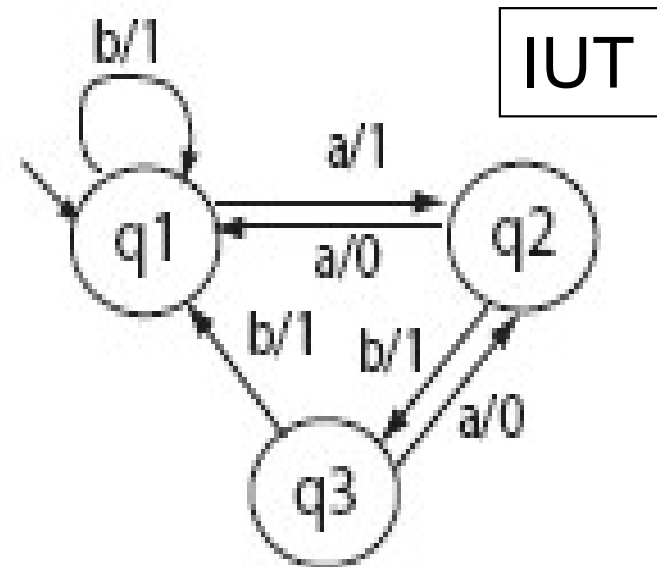
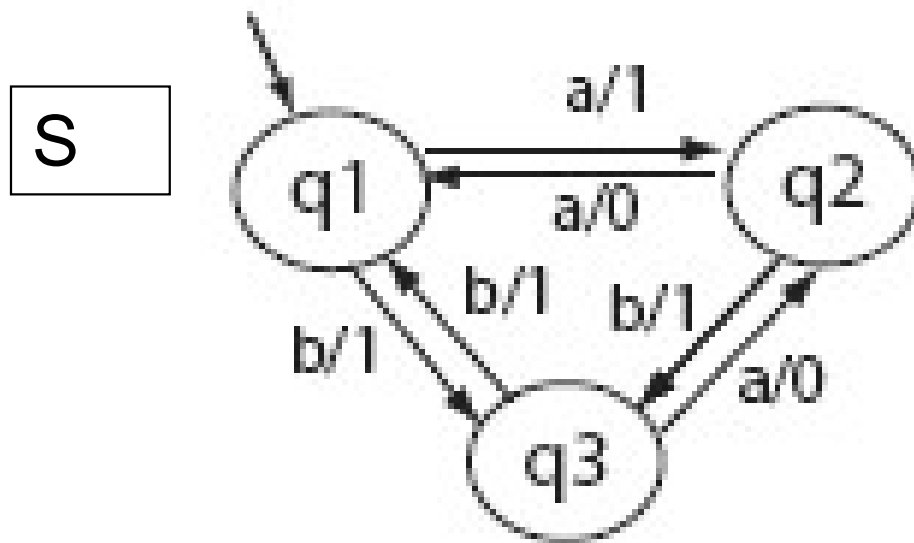
- Generate a test set T that distinguishes all wrong implementation with an extra state.



Hint. Use W and then Wp

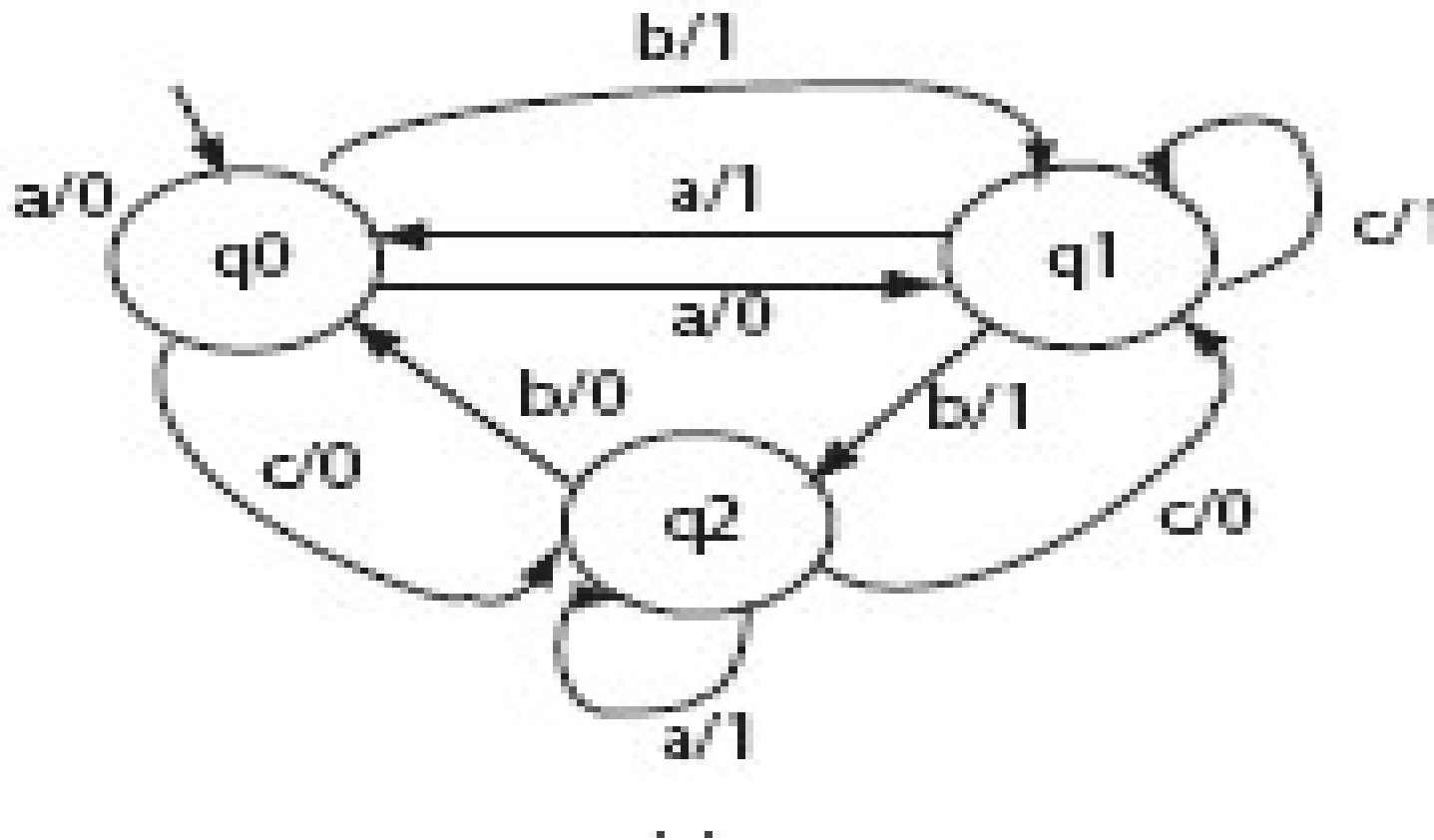
Exercise

- Construct the characterization set W and the transition cover for the machine S
- Using the W -method construct set Z assuming that $m = 3$ and derive a test set T .
- Does any element of T reveal the transfer error in the IUT?



Exercise TwTwp (1/3)

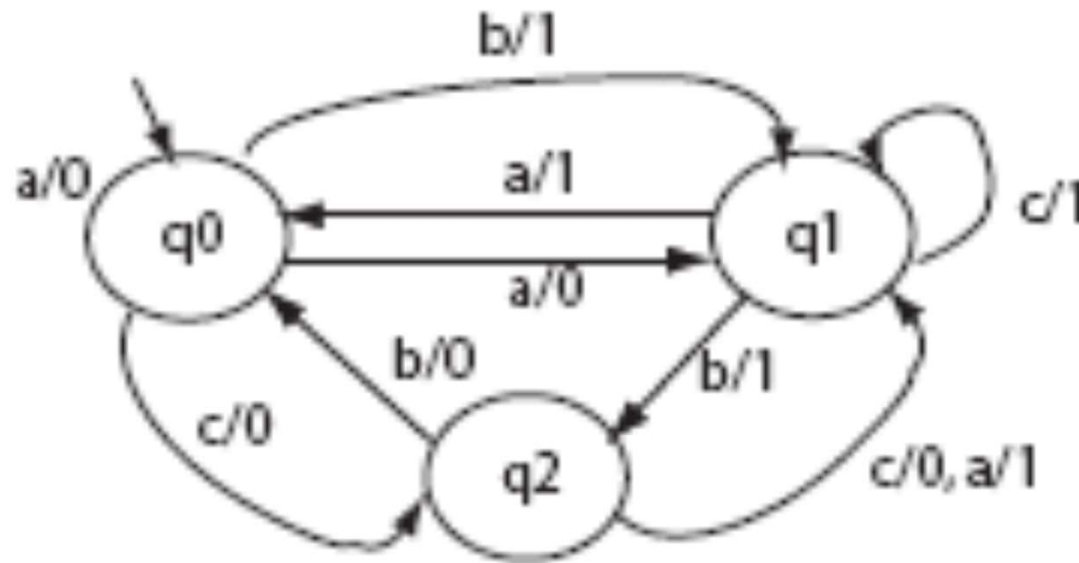
- Derive a transition cover set P , the state cover set S , the characterization set W , and state identification sets
- Derive a test set Tw and a test Twp .
- Compare the sizes of the two sets.



Exercise TwTwp (2/3)

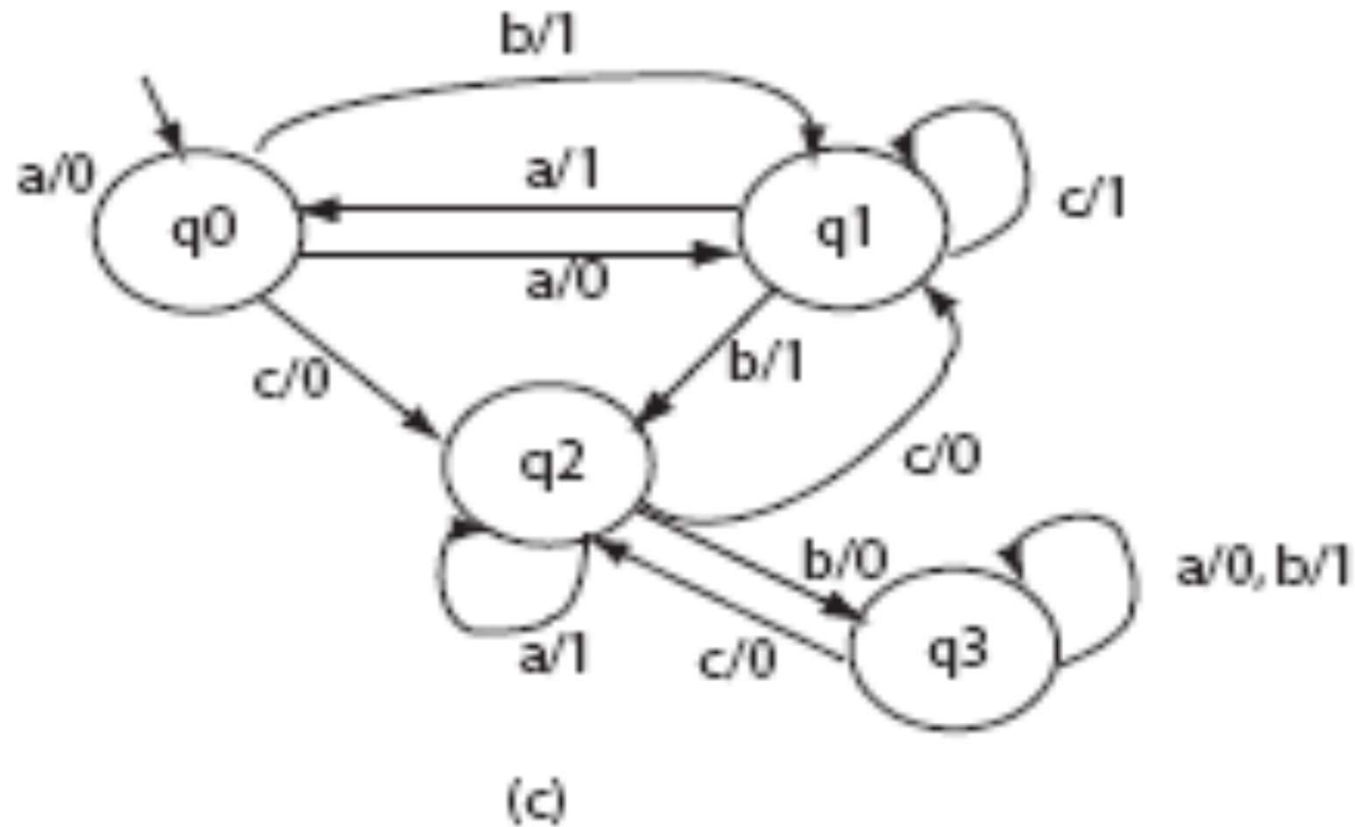
It contains a transfer error in state q_2 . Which tests in Tw and Twp reveal this error ?

It



Exercise TwTwp (3/3)

- It contains an extra state q_3 , and transfer error in state q_2 . Which tests in Tw and Twp reveal this error?



Exercise. Aditya Wp, $m=n$ (1/?)

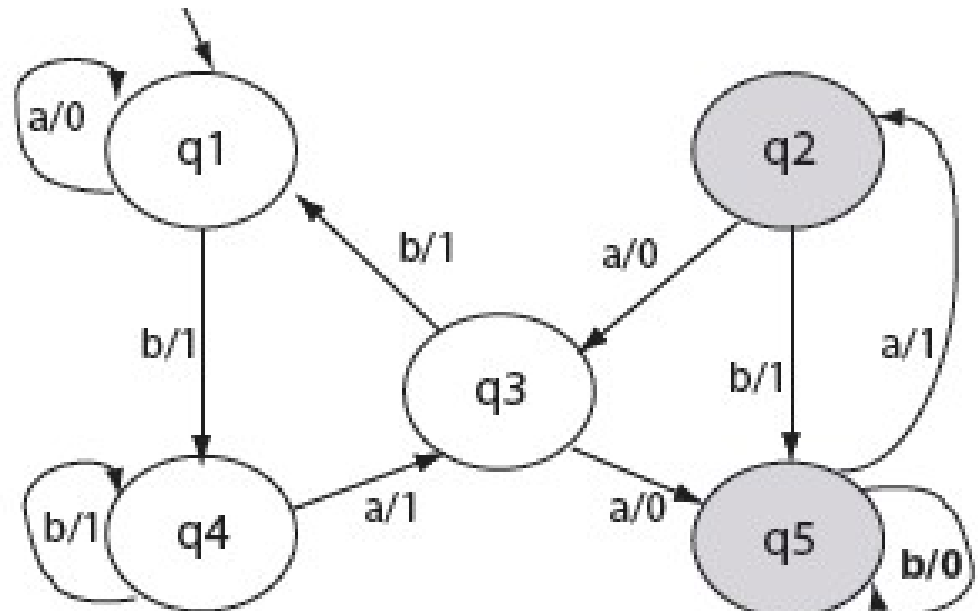
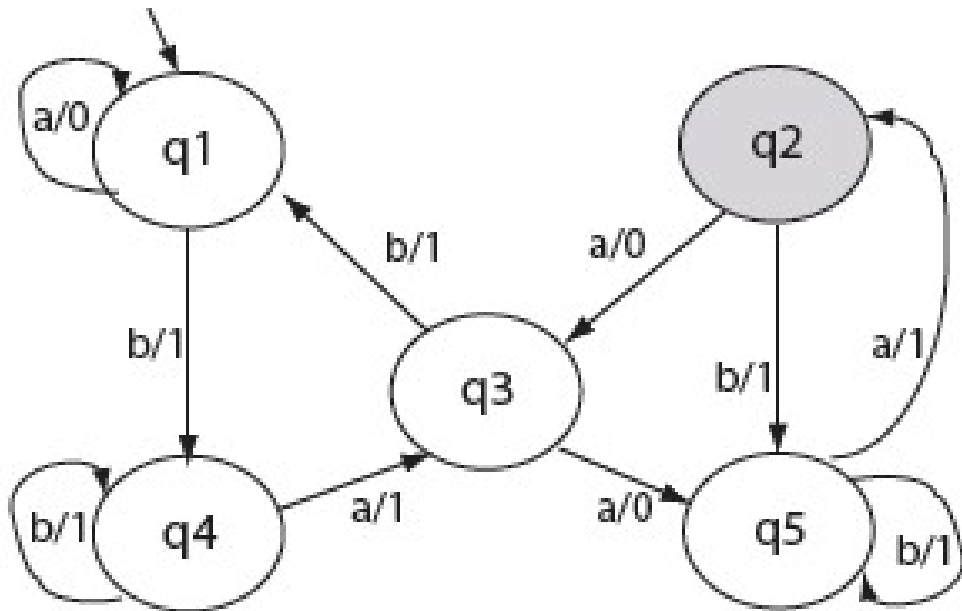
- Generate tests using the Wp-method for the specification Aditya. No extra states in IUT

Exercise. Aditya Wp, m=n (2/?)

- Answ.
 $W = \{a, aa, aaa, baaa\}$
 $P = \{\epsilon, a, b, bb, ba, bab, baa, baab, baaa, baaab, baaaa\}$
 $S = \{\epsilon, b, ba, baa, baaa\}$
 $W1 = \{baaa, aa, a\}$
 $W2 = \{baaa, aa, a\}$
 $W3 = \{a, aa\}$
 $W4 = \{a, aaa\}$
 $W5 = \{a, aaa\}$
- $T1 = S.W = \{a, aa, aaa, baaa, ba, baa, baaa, bbaaa, baa, baaa, baaa, babaaa, baaa, baaaa, baaaaa, baabaaa, baaaa, baaaaa, baaaaaa, baaabaaa\}$
- $R = P - S = \{a, bb, bab, baab, baaab, baaaa\}$.
- $T(q1, a) = q1, T(q1, bb) = q4, T(q1, bab) = q5, T(q1, baab) = q5, T(q1, baaab) = q5, T(q1, baaaa) = q1$
- $T2 = R \otimes W = (\{a\}.W1) \cup (\{bb\}.W4) \cup (\{bab\}.W5) \cup (\{baab\}.W5) \cup (\{baaab\}.W5) \cup (\{baaaa\}.W1)$
- $= \{abaaa, aaa, aa, bba, bbaaa, baba, babaaa, baaba, baabaaa, baaaba, baaabaaa, baaaabaaa, baaaaaa, baaaaa\}$
- $T = T1 \cup T2$.
 T contains a total of 34 tests, with 20 from $T1$ and 14 from $T2$.
This is in contrast to the 44 tests generated using the W-method when m=n

Exercise. Aditya Wp, $m=n$ (3/?)

Use T to test the two IUTs below



Exercise. Aditya Wp, $m=n+1$ (1/?)

Construct a test set using the Wp-method for machine Aditya given that the corresponding IUT contains an extra state.

Exercise. Aditya Wp, $m=n+1$ (2/?)

Answ. $n=5$, $m=6$.

- $X = \{a, b\}$ $W = \{a, aa, aaa, baaa\}$ $P = \{\epsilon, a, b, bb, ba, baa, bab, baab, baaa, baaab, baaaa\}$ $S = \{\epsilon, b, ba, baa, baaa\}$ $W1 = \{baaa, aa, a\}$ $W2 = \{baaa, aa, a\}$ $W3 = \{a, aa\}$ $W4 = \{a, aaa\}$ $W5 = \{a, aaa\}$
- $T1 = (S.W) \cup (S.X.W)$
 - $S.W = \{a, aa, aaa, baaa, ba, baa, baaa, bbaaa, baa, baaa, baaaa, babaaa, baaa, baaaa, baaaaa, baabaaa, baaaaa, baaaaa, baaaaaa, baaabaaa\}$
 - $S.X = \{a, b, ba, bb, baa, bab, baaa, baab, baaaa, baaab\}$
 - $S.X.W = \{aa, aaa, aaaa, abaaa, ba, baa, baaa, bbaaa, baa, baaa, baaaa, babaaa, bba, bbaa, bbaaa, bbbaaa, baaa, baaaa, baaaaa, baabaaa, baba, babaa, babaaa, babbaaa, baaaa, baaaaa, baaaaaa, baaabaaa, baaba, baabaa, baabaaa, baabbaaa, baaaaa, baaaaaa, baaaaaaa, baaaaaaa, baaaabaaa, baaaba, baaabaa, baaabaaa, baaabaaa\}$

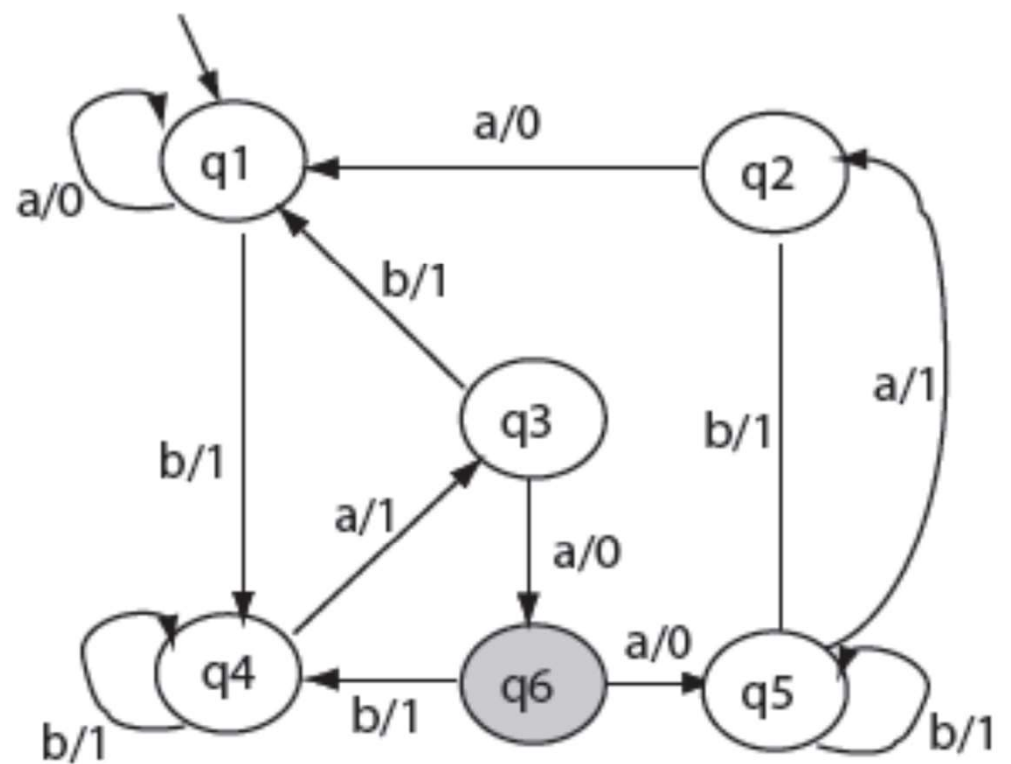
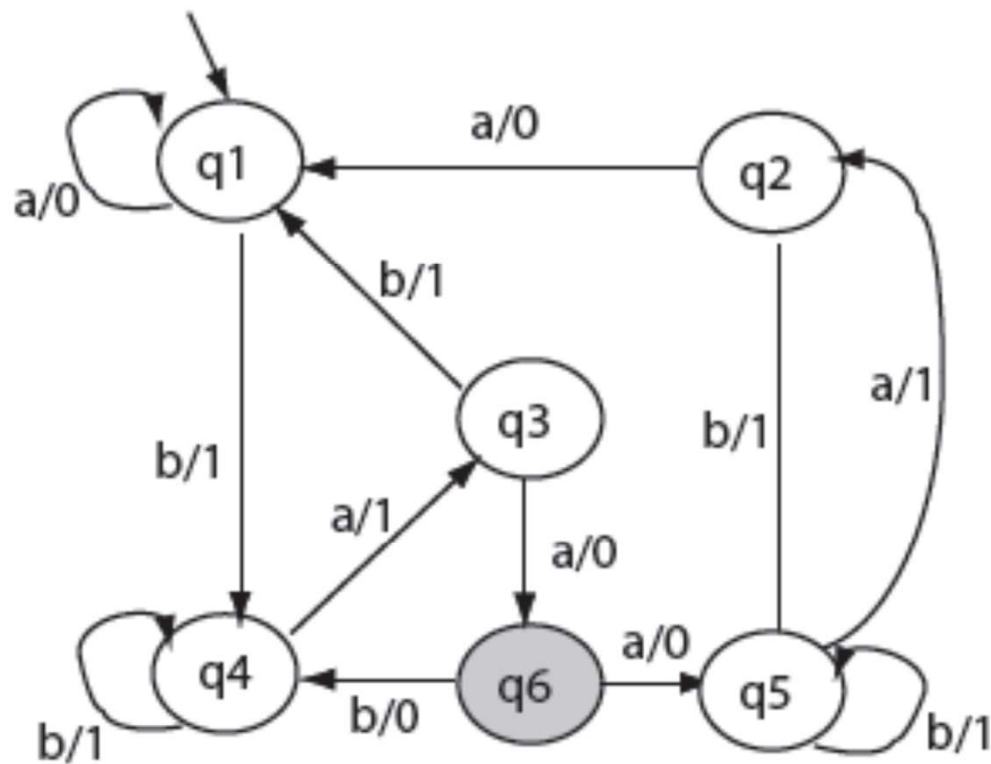
$T1$ contains a total of 60 tests of which 20 are in $S.W$ and 40 in $S.X.W$.

Exercise. Aditya Wp, $m=n+1$ (3/?)

- $T2 = (R \otimes W) \cup (R.X \otimes W)$
- $R \otimes W = (\{a\}.W1) \cup (\{bb\}.W4) \cup (\{baab\}.W5) \cup \{bab\}.W1) \cup \{baaab\}.W1 \cup \{baaaa\}.W5 = \{abaaa, aaa, aa, bba, bbaaa, baaba, baabaaa\}$
- $R.X \otimes W = (aa.W1) \cup (ab.W4) \cup (bba.W3) \cup (bbb.W4) \cup (baaba.W2) \cup (baabb.W5) \cup (baba.W1) \cup (babb.W4) \cup (baaaba.W2) \cup (baaabb.W5) \cup (baaaaa.W1) \cup (baaaab.W5) = \{aabaaa, aaaa, aaa, aba, abaaa, bbaa, bbaaa, bbba, bbbaaa, baababaaa, baabaaa, baabaa, baabba, baabbaaa\}$

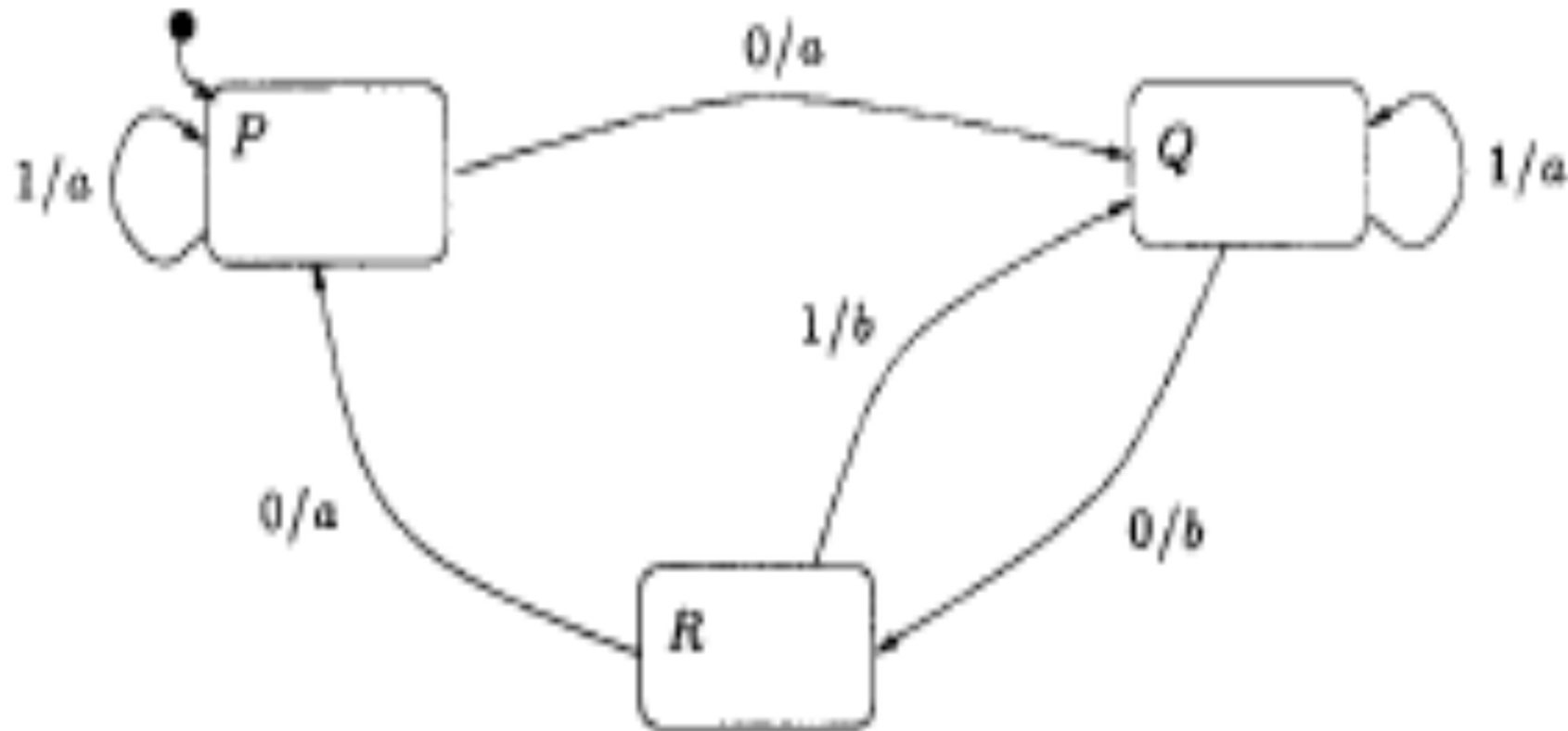
Exercise. Aditya Wp, $m=n+1$ (4/?)

Use T to test the two implementations below



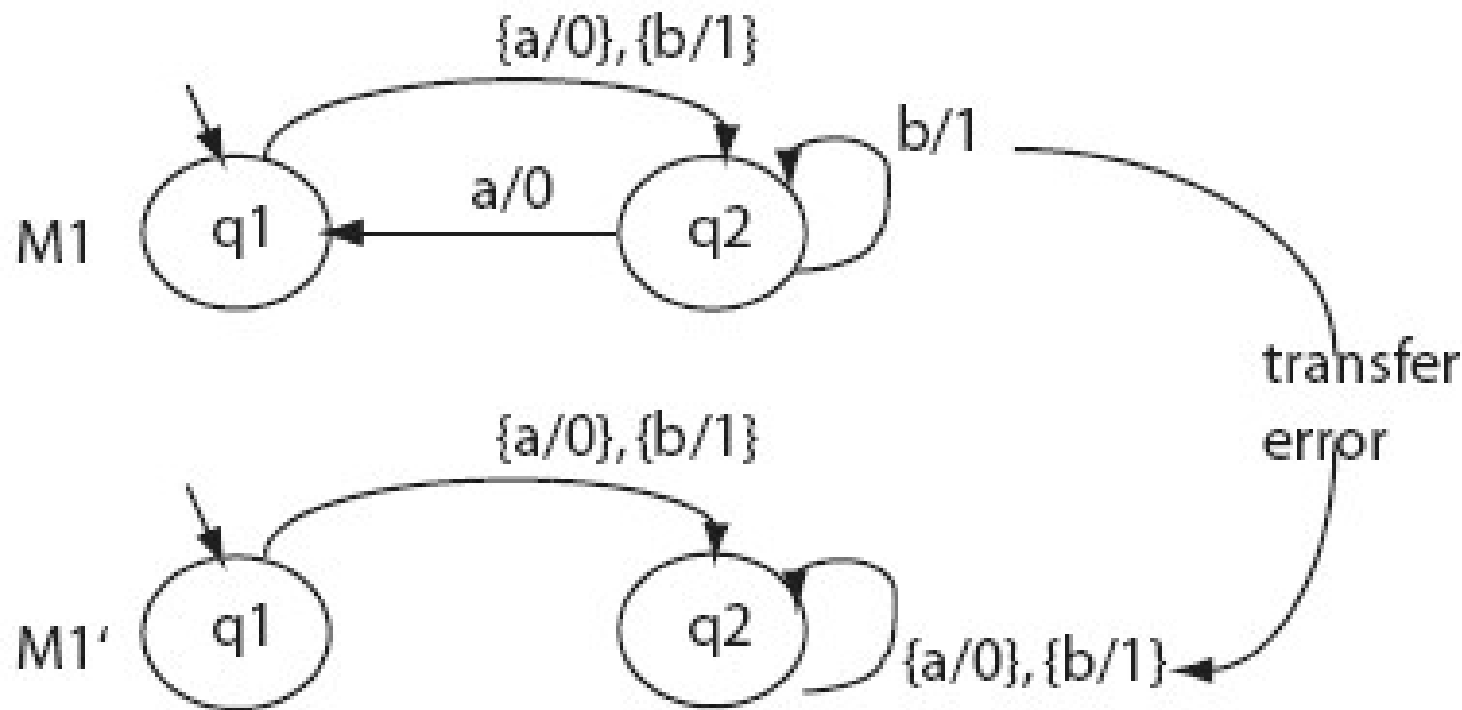
Exercise (Sadeghipour) (1/2)

Construct the Tw test set



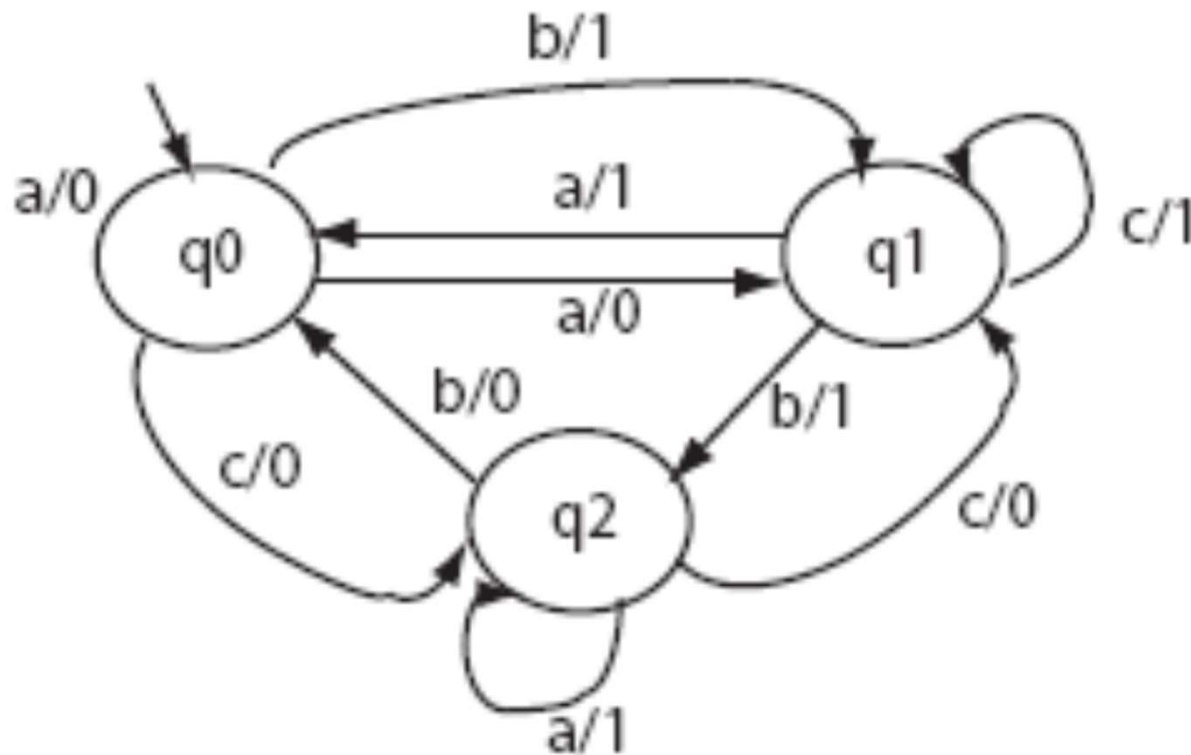
W@Wp all errors

- Derive tests using the W-method, and then using the Wp-method, and show that each test set derived detects all errors in the IUT M1'

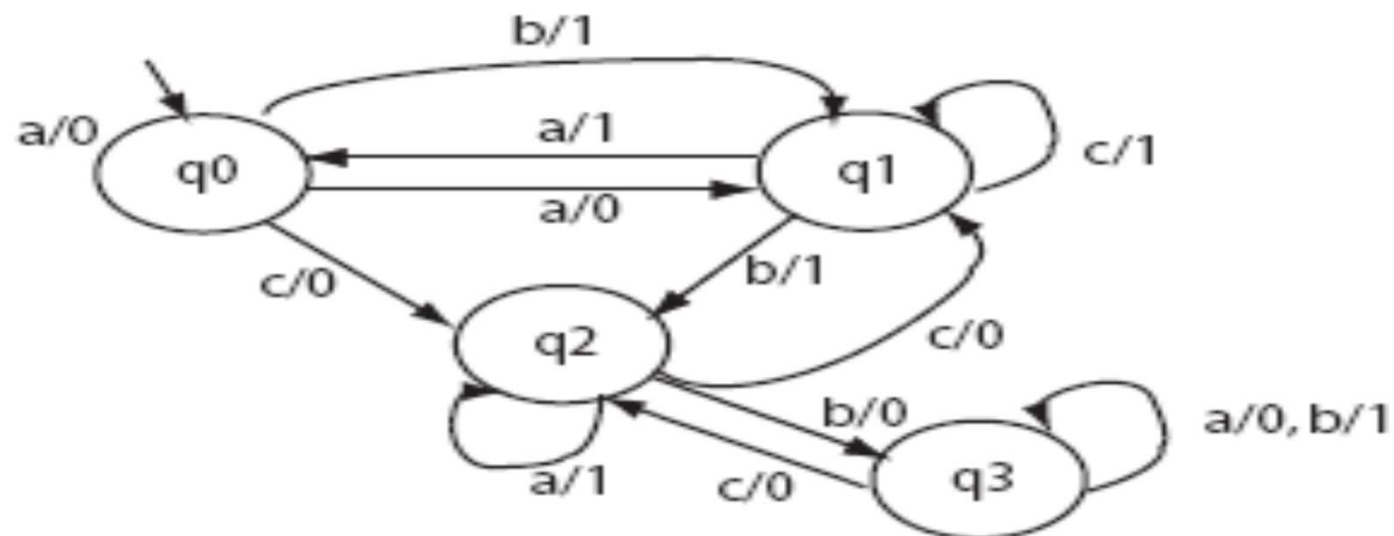
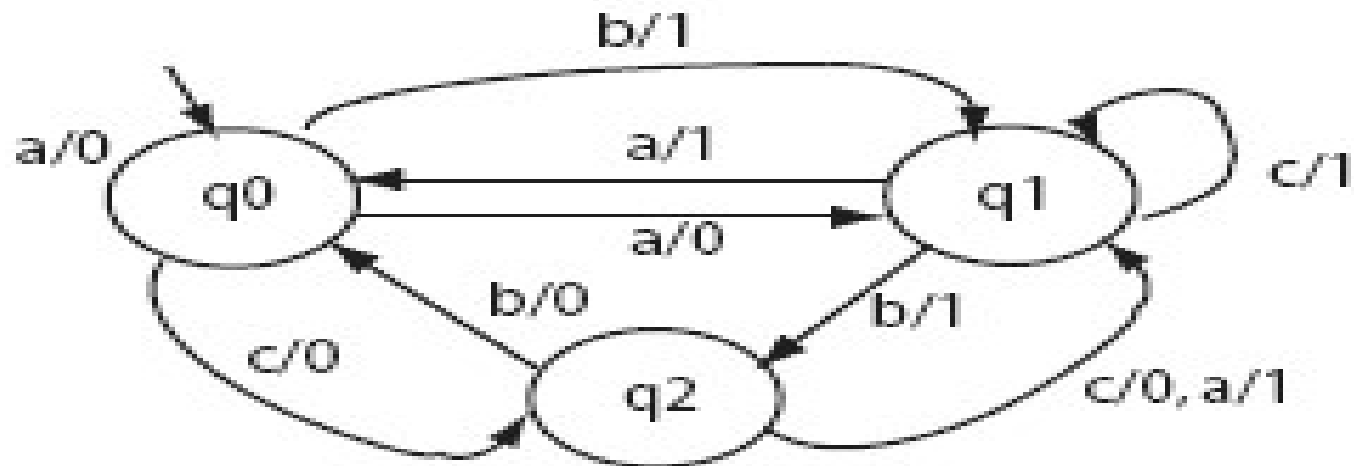


Exercise. W or WP (1/2)

- Derive tests for the specification below, taking into consideration each of the IUTs from the next slide



Exercise. W or WP (2/2)



Exercise (Sadeghipour) (2/2)

- Apply Tw to discover the errors in this implementation

